iCard

Accepting e-commerce payments for merchants

IPG API BM Gambling

Protocol Version 4.5

Document version 3

Table of Contents

Version control

Document version	Protocol version	Author	Description	Date posted
1	4.2	Veronika Vasileva	Description of Google Pay integration for Gambling merchants in IPG version 4.2 in all methods properties and JS SDK	30.03.2024
2	4.5	Veronika Vasileva	New Callbacks functionality – initial decription	15.04.2025
3	4.5	Veronika Vasileva	Full description for Business model Gambling and all type of implementation and summary with changes between protocol 4.3 and 4.5	10.06.2025

Summary of the new features

Here's an overview of the differences between protocol versions **IPG 4.3** and **IPG 4.5**:

New signature: New algorithm for signature calculation

Notification Method: The Notify methods have been removed. Instead, a JSON-formatted callback informs about the final transaction status.

Behavior after failed notification: The reversal execution has been removed if the merchant does not respond with OK to the notification. Instead, the notification is repeated until the merchant responds with a 200 OK status.

Redirect Behavior: The reverse redirect to the merchant's OK or Cancel page now uses the GET method, and no data is sent.

Token Encryption: The requirements for token encryption has been removed when calling the following methods: IPG3DSPurchaseWithStoredCard, IPGPaymentToken, IPGOCT.

Cart Items Requirement: The need to send CartItems has been removed.

FieldsOrder Field: This field has been eliminated from backend method responses, affecting only integrations using versions 4.3 and 4.4.

New Field – PostResultAction: Introduced specifically for redirect integrations, it defines the behavior after an operation is completed.

Error Codes and Messages: New error codes and messages have been added.

Embedded checkout: Support for widget Integrations has been removed. Instead, embedded payment should be used.

IPG3DSPurchaseWithStoredCard method: Removed mandatory requirements for billing address data.

IPGReversal method: Added new fields for OrderID and MID.

OrderID: Change of the length to 50 characters**.**

CardToken: Parameter Token has been changed to CardToken for methods - IPG3DSPurchaseWithStoredCard and IPGOCT.

IPGOCT method: Change response parameters.

Security and availability

Connection between Merchant and iCARD is handled through internet using HTTPS protocol (SSL over HTTP). Requests and responses are digitally signed both. iCARD host is located at tier IV datacenter in Luxembourg. Public address for IPG is BGP enabled and available through all first level internet providers.

Exchange folder for partners (if needed) is located at a SFTP server which enables encrypted file sharing between parties. The partner receives the account and password for the SFTP directory via fax, email or SMS.

iCARD supplies an emergency support line via e-mail or phone which is 7x24 enabled and reaches certified engineers.

Introduction

This document describes the interface for e-commerce payments via payment gateway. The Merchant should integrate the iPayment Gateway API (IPG API) at the site accepting card payments. IPG API will gain access to the entry point of iPayment Gateway (IPG) managed by iCard AD (iCARD). IPG will handle and guide the cardholder during the payment process, will check the card sensitive data and will process a payment transaction through card schemes (VISA, MasterCard).

IPG API will provide:

- Secured page and Secured communication channel with the Merchant
- Storing of merchant private data (amount, payment methods, transaction details etc.)
- Financial transactions to VISA, MasterCard – transparent for the Merchant
- Operations for the front-end: Purchase transaction
- Operations for the back-end: Refund, Reversal
- 3D processing

Out of scope for this document:

- Merchant statements and payouts
- Merchant back-end (iMerchant)
- Merchant support system (IPGPlatform)

The purpose of this document is to specify the IPG API Interface and demonstrate how it is used in the most common way.

All techniques used within the interface are standard throughout the industry and should be very easy to implement on any platform.

Test IPG API

iCard's procedure is as follows: Once the integration process is started there is a technical meeting in which we specify the methods for sandbox and needed settings.

From the iCard team the merchant receives a kit including files with base settings for sandbox environment to start the integration process. During the integration, there is channel created for support in which the merchant can ask questions and receive answers about the integration.

When iCard receives an indication from merchant's side that they are ready with the integration:

1. Based on the commands you have integrated, iCard will provide a sandbox test scenario.

2. Merchant has to execute it and fill in the provided TRN/OrderIDs, and then return it back to iCard for QA check.

3. After validation that the tests have been passed correctly, you will receive a kit with settings and test scenario for the Production environment.

4.Upon passing the Production tests, the MID – merchant ID will be forbidden until the clearing is passed successfully on the next day.

A few important notes:

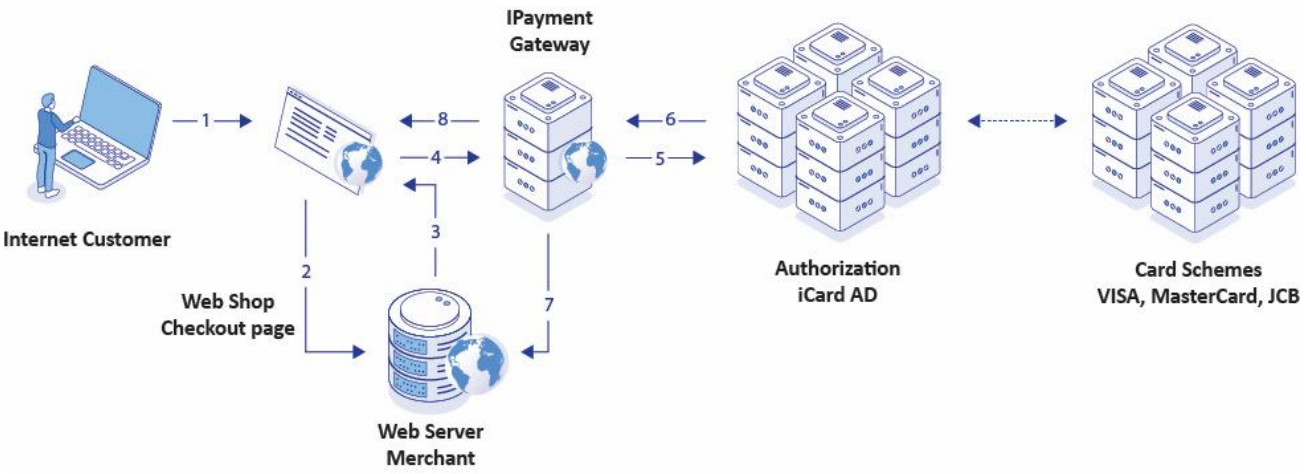
It is best practice to begin the sandbox tests once you are also sure about your live date for Production environment and traffic flow, as if there is a gap between the sandbox tests and production tests longer than 1 month, you will have to pass the sandbox tests again.

In addition, it is recommended to have a period for tests with "Family & Friends" after the production tests.

Additionally, we kindly request that a test account be created for us on your platform to allow ongoing monitoring and testing.

Accepting e-commerce payments with IPG interface

Overview



1. Internet customer at web shop checkout page
2. Payment initiated by customer.
3. Merchant web server initiates payment through IPG. Merchant web server should redirect the browser to IPG web address.
4. Customer web browser is redirected to IPG web page.
5. Customer is requested to input the card data and press PAY. IPG handles the 3D secure processing and financial transaction messaging.
6. IPG receives the details for the payment – successful or declined.
7. IPG passes the result to Merchant Web Server.
8. IPG redirects to Web Shop “checkout result” page.

HTTP POST

Data transfer between Merchant and IPG is made by HTTP POST. All the parameters for the requests are in the body in [parameter=value] form. Separator between tokens is [&]. The body is URL Encoded. Character encoding is UTF-8.

```
Example:
POST /sandbox/ HTTP/2
Host: dev-ipg.icards.eu
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:123.0) Gecko/20100101 Firefox/123.0
Content-Length: 793
Content-Type: application/x-www-form-urlencoded
IPGmethod=IPGPurchase&KeyIndex=1&KeyIndexResp=1&IPGVersion=4.2&Language=en&Originator=33…
```

Data Type Formats

Data Type in document	Description	Example
int	integer	1
String	string	This is a string
Date	ISO 8601 date string YYYY-MM-DD	2021-09-14
DateTime	ISO 8601 datetime string YYYY-MM-DD HH:mm:SS	2021-09-14 23:59:59
A (n)	Alpha string. [n] characters required	Alpha string
AN (n)	Alphanumeric string. [n] characters required	Alphanumeric string
N (n)	Numeric string. [n] characters required. Number is left-padded with zeroes.	000123
Double (M, D)	Numeric string with decimal point. Only point is used (no commas or other characters for decimal point). Here, (M,D) means than values can be accepted with up to M digits in total, of which D digits may be after the decimal point.	34.56
BASE64	Sting used to pass binary data. The binary data should be converted to base64 standard.	YW55IGNhcm5hbCBwbGVhc3VyZQ==
XML	Simple in place XML array.	<?xml version="1.0"?> <ipg_response> <status>0</status> <status_msg>Success</status_msg> …… </ipg_response>

JSON	JSON string	{
		“Field1”: “value”,
		“Field2”: “value”
		}

Signatures

Overview

The data communication between the merchant's web service and the iCard payment platform is protected by using the TLS (Transport Layer Security) protocol version 1.2 or later. This ensures the confidentiality of the data being transmitted, though the protocol cannot guarantee the message integrity and ensure that the message author possesses the private key. Therefore, every message must be digitally signed by using the private key.

For signing process, both iCARD and the Merchant generate public and private key pairs and exchange the public keys. Key pairs are generated using RSA algorithm. Every of the parties are using the private key to sign the message and the opposite side authenticate the sender with corresponding public key.

For generation of keys pair you can use this link: Generate RSA Keys

Regardless of the interface that is used for working with the payment platform, digital signatures should be included in all requests, callbacks, and responses. Thus, before sending any request to the platform, a signature should be generated and included in the request to be sent; and in case of receiving responses and callbacks from the platform, it is necessary to verify the received data by comparing the signatures to the ones generated on the merchant side.

This section describes the algorithms for the digital signature generation and the data integrity verification, including examples with the use of these algorithms and interactive forms for testing the workflows using signatures.

Signature generation with IPG version greater than or equal to 4.5

Signing algorithm

The algorithm input includes the following:

- **Data to sign** - Generally, this is the request body, all request parameters without the signature.
- **Private key – key for generation**

Thus, the algorithm includes the following steps:

1. Input validation. Make sure the following requirements are met:

    1. Data to sign does not contain any signature parameter even if it is empty.

    2. The private key is readily available.

Thus, the algorithm includes the following steps:

2. Conversion of all the strings to UTF-8 with parameters sorted in natural order. Complete the following steps:

- Change case to lower for all keys: IPGmethod -> ipgmethod
- Encode any Boolean values as follows: replace false with 0, replace true with 1. Mind that this rule applies only to Boolean values. If any string parameter contains "false" or "true" string value, the value is not replaced with 0 or 1 but is treated as any other string value.
- Convert each parameter into a string that contains the full path to the parameter (with all its parents), parameter name, and parameter value:

```
<parent_1>:...:<parent_n>:<parameter_name>:<parameter_value>
```

The parents are the object(s) and/or arrays in which the parameter is contained. Parents are ordered by embedding level starting with the topmost one.

The parent names, parameter name, and parameter value are delimited by colon (:); delete any commas between “key-value” pairs and any quotation marks that delimit string values.

- Leave any empty parameter values empty. In other words, do not replace any empty values with blank space or null. For instance, "payment_description":"" is replaced with payment_description:.
- Add index numbers to array elements starting with zero, for instance ["alpha", "beta", "gamma"] is replaced with three strings: 0:alpha, 1:beta, and 2:gamma.
- Empty arrays are totally ignored and not included in the string set used to generate the signature.
- Convert all the strings to UTF-8.
- Sort the strings in natural sort order and join them in a single string by using semicolon (;) as a delimiter.

3. Calculate signature with SHA-256 algorithm

4. Encode signature by using the Base64 scheme.

5. Add the Signature parameter to the request body using the signature from the previous step as its value.

Example with IPG version greater than or equal to 4.5

Initial data:

```
[
    "IPGmethod" => "IPGPurchase",
    "KeyIndex" => "1",
    "KeyIndexResp" => "1",
    "IPGVersion" => "4.5",
    "Language" => "en",
    "Originator" => "33",
    "BannerIndex" => "1",
    "MID" => "000000000000113",
    "Currency" => "975",
    "MIDName" => "IPG TEST 4.5",
    "CustomerIP" => "127.0.0.1",
    "OrderID" => "8A540554-1551-4533-B246-42CAD55EE8DE",
    "CustomerIdentifier" => "SZ-1868",
    "BoolExample" => true,
    "EmptyExample" => ""
    “Signature” => “<signature that needs to be generated>”
]
```

Make sure there is no signature parameter in your request even it is empty:

```
[
    "IPGmethod" => "IPGPurchase",
    "KeyIndex" => "1",
    "KeyIndexResp" => "1",
    "IPGVersion" => "4.5",
    "Language" => "en",
    "Originator" => "33",
    "BannerIndex" => "1",
    "MID" => "000000000000113",
    "Currency" => "975",
    "MIDName" => "IPG TEST 4.5",
    "CustomerIP" => "127.0.0.1",
    "OrderID" => "8A540554-1551-4533-B246-42CAD55EE8DE",
    "CustomerIdentifier" => "SZ-1868",
    "BoolExample" => true,
    "EmptyExample" => ""
    ~~“Signature” => “<signature that needs to be generated>”~~
]
```

Change key case to lower

```
[
    "ipgmethod" => "IPGPurchase",
    "keyindex" => "1",
    "keyindexresp" => "1",
    "ipgversion" => "4.5",
    "language" => "en",
    "originator" => "33",
    "bannerindex" => "1",
    "mid" => "000000000000113",
    "currency" => "975",
    "midname" => "IPG TEST 4.5",
    "customerip" => "127.0.0.1",
    "orderid" => "8A540554-1551-4533-B246-42CAD55EE8DE",
    "customeridentifier" => "SZ-1868",
    "boolexample" => true,
    "emptyexample" => ""
]
```

Convert all strings to UTF-8 as per algorithm description:

```
[
```

```
    "ipgmethod:IPGPurchase"
    "keyindex:1"
    "keyindexresp:1"
    "ipgversion:4.5"
    "language:en"
    "originator:33"
    "bannerindex:1"
    "mid:000000000000113"
    "currency:975"
    "amount:1.00"
    "midname:IPG TEST 4.5"
    "customerip:127.0.0.1"
    "orderid:8A540554-1551-4533-B246-42CAD55EE8DE"
    "customeridentifier:SZ-1868"
    "boolexample:1"
    "emptyexample:
]
```

Sort the strings in natural sort order:

```
[
    "amount:1.00",
    "bannerindex:1",
    "boolexample:1",
    "currency:975",
    "customeridentifier:SZ-1868",
    "customerip:127.0.0.1",
    "emptyexample:",
    "ipgmethod:IPGPurchase",
    "ipgversion:4.5",
    "keyindex:1",
    "keyindexresp:1",
    "language:en",
    "mid:000000000000113",
    "midname:IPG TEST 4.5",
    "orderid:8A540554-1551-4533-B246-42CAD55EE8DE",
    "originator:33"
]
```

Join all strings into a single string by using semicolon (;) as a delimiter:

"amount:1.00;bannerindex:1;boolexample:1;currency:975;customeridentifier:SZ-1868;customerip:127.0.0.1;emptyexample:;ipgmethod:IPGPurchase;ipgversion:4.5;keyindex:1;keyindexresp:1;language:en;mid:000000000000113;midname:IPG TEST 4.5;orderid:8A540554-1551-4533-B246-42CAD55EE8DE;originator:33"

Get private key:

PHP

```
$privateKey = openssl_get_privatekey(privateKeyString);
```

C#

```
var privateCert = File.ReadAllText(privateKeyString);
var key = RSA.Create();
key.ImportFromPem(privateCert.ToCharArray());
```

Calculate signature with SHA-256 algorithm:

PHP

```
openssl_sign($dataToSign, $signature, $privateKey, OPENSSL_ALGO_SHA256);
```

C#

```
var sha = SHA256.Create();
var signature = key.SignHash(sha.ComputeHash(Encoding.UTF8.GetBytes(dataToSign)),
HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);
```

Encode to Base64

PHP

```
$base64Signature = base64_encode($signature);
```

```
C#
var base64Signature = Convert.ToBase64String(signature);
```

Add the resulted signature to initial data:

```
[
    "IPGmethod" => "IPGPurchase",
    "KeyIndex" => "1",
    "KeyIndexResp" => "1",
    "IPGVersion" => "4.5",
    "Language" => "en",
    "Originator" => "33",
    "BannerIndex" => "1",
    "MID" => "000000000000113",
    "Currency" => "975",
    "MIDName" => "IPG TEST 4.5",
    "CustomerIP" => "127.0.0.1",
    "OrderID" => "8A540554-1551-4533-B246-42CAD55EE8DE",
    "CustomerIdentifier" => "SZ-1868",
    "BoolExample" => true,
    "EmptyExample" => ""
    “Signature” => “PNYhiEtXvwTB2ixMID+hYuJIc7+VUlYcQzyH9xXTSGm2K7NiSNBe9oYeyv0Bi0e==”
]
```

Signature verification

Verification algorithm

The algorithm input includes the following:

1. **Signed data to verify** – As a rule, it is a callback or response body in the JSON format with the signature parameter.
2. **Public key** – Public key for validation the signature

The algorithm description, the examples, and test forms below use the most common algorithm implementation that includes callback or response body in JSON format as input (without the signature parameter) and output (with the signature parameter) which is the result of signature verification.

Thus, the algorithm includes the following steps:

1. Input validation - Make sure the following requirements are met:

- Data conforms to the JSON format.
- Data to sign contains a signature parameter with the signature value.
- The public key is readily available.

2. Extracting signature from the data to verify - Store the value of the signature parameter value for further reference and remove the parameter from the input data. Decode signature value from Base64 scheme.

3. Generate signature for the data to verify. Complete steps 2 as specified in the signature generation algorithm description:

4. Verify signatire with SHA-256 algorithm

Signature verification example with IPG version greater than or equal to 4.5

Callback body:


```
{
    “CardData”: {
            "Pan": "532610***0004",
            "Type": "MasterCard",
            "CardholderName": "Test",
            "ExpMonth": "06",
            "ExpYear": "25"
    },
    “Customer”: {
            "Email": "payment.test@test.com",
            "Identifier": "SZ-1868",
            "IPAddress": "127.0.0.1"
    },
    “Payment”: {
            "OrderId": "8A6E3069-E3B2-4EB7-976F-5A6319004F26",
            "MID": "000000000000112",
            "Date": "2025-05-12T18:04:21+03:00",
            "Type": "IPGPurchase",
            "Context": "CardPay",
            "Status": "success",
            "Sum": {
                    "Amount": "1.00",
                    "Currency": 978
```

```
            },
            "Interface": "redirect"
    },
    “Operation”: {
            "Type": "authorization",
            "Status": "success",
            "Date": "2025-05-12T18:04:21+03:00",
            "Code": 0,
            "Message": "Success",
            "Sum": {
                        "Amount": "1.00",
                        "Currency": 978
            },
            "Provider": {
                        "Trn": "20250512150419228574",
                        "Date": "2025-05-12T18:04:19+03:00",
                        "RespCode": "00",
                        "Approval": "CA0DD8"
            },
            "StoreCard": {
                        "CardToken": "40B1B011C4A21EA65A8AA06E9D767ECE348ADB2E2D4E4E6C3A0536E452619059"
            },
            "Eci": "05"
    },
    Signature => n+kUHOgJudQ2bz1lnp80T5V0i9FXLOaBguzheG07vVoEO97UDjuTR==
}
```


Remove the signature parameter and its value from the callback:


```
{
    “CardData”: {
            "Pan": "532610***0004",
            "Type": "MasterCard",
            "CardholderName": "Test",
            "ExpMonth": "06",
            "ExpYear": "25"
    },
    “Customer”: {
            "Email": "payment.test@test.com",
            "Identifier": "SZ-1868",
            "IPAddress": "127.0.0.1"
    },
    “Payment”: {
            "OrderId": "8A6E3069-E3B2-4EB7-976F-5A6319004F26",
            "MID": "000000000000112",
            "Date": "2025-05-12T18:04:21+03:00",
            "Type": "IPGPurchase",
            "Context": "CardPay",
            "Status": "success",
            "Sum": {
                        "Amount": "1.00",
                        "Currency": 978
            },
            "Interface": "redirect"
    },
    “Operation”: {
            "Type": "authorization",
            "Status": "success",
            "Date": "2025-05-12T18:04:21+03:00",
            "Code": 0,
            "Message": "Success",
            "Sum": {
                        "Amount": "1.00",
                        "Currency": 978
```

```
        },
        "Provider": {
                "Trn": "20250512150419228574",
                "Date": "2025-05-12T18:04:19+03:00",
                "RespCode": "00",
                "Approval": "CA0DD8"
        },
        "StoreCard": {
                "CardToken": "40B1B011C4A21EA65A8AA06E9D767ECE348ADB2E2D4E4E6C3A0536E452619059"
        },
        "Eci": "05"
    },
    Signature => n+kUHOgJudQ2bz1lnp80T5V0i9FXLOaBguzheG07vVoEO97UDjuTR==
}
```


Change key case to lower:


```
{
    “carddata”: {
        "pan": "532610***0004",
        "type": "MasterCard",
        "cardholdername": "Test",
        "expmonth": "06",
        "expyear": "25"
    },
    “customer”: {
        "email": "payment.test@test.com",
        "identifier": "SZ-1868",
        "ipaddress": "127.0.0.1"
    },
    “payment”: {
        "orderid": "8A6E3069-E3B2-4EB7-976F-5A6319004F26",
        "mid": "000000000000112",
        "date": "2025-05-12T18:04:21+03:00",
        "type": "IPGPurchase",
        "context": "CardPay",
        "status": "success",
        "sum": {
                "amount": "1.00",
                "currency": 978
        },
        "interface": "redirect"
    },
    “operation”: {
        "type": "authorization",
        "status": "success",
        "date": "2025-05-12T18:04:21+03:00",
        "code": 0,
        "message": "Success",
        "sum": {
                "amount": "1.00",
                "currency": 978
        },
        "provider": {
                "trn": "20250512150419228574",
                "date": "2025-05-12T18:04:19+03:00",
                "respcode": "00",
                "approval": "CA0DD8"
        },
        "storecard": {
                "cardtoken": "40B1B011C4A21EA65A8AA06E9D767ECE348ADB2E2D4E4E6C3A0536E452619059"
        },
        "eci": "05"
    }
```

```
}
```

Convert all parameter strings to UTF-8 as per algorithm description:

```
"carddata:pan:532610***0004"
"carddata:type:MasterCard"
"carddata:cardholdername:Test"
"carddata:expmonth:06"
"carddata:expyear:25"
"customer:email:payment.test@test.com"
"customer:identifier:SZ-1868"
"customer:ipaddress:127.0.0.1"
"payment:orderid:8A6E3069-E3B2-4EB7-976F-5A6319004F26"
"payment:mid:000000000000112"
"payment:date:2025-05-12T18:04:21+03:00"
"payment:type:IPGPurchase"
"payment:context:CardPay"
"payment:status:success"
"payment:sum:amount:1.00"
"payment:sum:currency:975"
"operation:type:authorization"
"operation:status:success"
"operation:date:2025-05-12T18:04:21+03:00"
"operation:code:0"
"operation:message:Success"
"operation:sum:amount:1.00"
"operation:sum:currency:975"
"operation:provider:trn:20250512150419228574"
"operation:provider:date:2025-05-12T18:04:19+03:00"
"operation:provider:respcode:00"
"operation:provider:approval:CA0DD8"
"operation:storecard:cardtoken:40B1B011C4A21EA65A8AA06E9D767ECE348ADB2E2D4E4E6C3A0536E452619059"
"operation:eci:05"
```

Sort the strings in natural sort order:

```
"carddata:cardholdername:Test"
"carddata:expmonth:06"
"carddata:expyear:25"
"carddata:pan:532610***0004"
"carddata:type:MasterCard"
"customer:email:payment.test@test.com"
"customer:identifier:SZ-1868"
"customer:ipaddress:127.0.0.1"
"operation:code:0"
"operation:date:2025-05-12T18:04:21+03:00"
"operation:eci:05"
"operation:message:Success"
"operation:provider:approval:CA0DD8"
"operation:provider:date:2025-05-12T18:04:19+03:00"
"operation:provider:respcode:00"
"operation:provider:trn:20250512150419228574"
"operation:status:success"
"operation:storecard:cardtoken:40B1B011C4A21EA65A8AA06E9D767ECE348ADB2E2D4E4E6C3A0536E452619059"
"operation:sum:amount:1.00"
"operation:sum:currency:975"
"operation:type:authorization"
"payment:context:CardPay"
"payment:date:2025-05-12T18:04:21+03:00"
"payment:mid:000000000000112"
"payment:orderid:8A6E3069-E3B2-4EB7-976F-5A6319004F26"
"payment:status:success"
```

```
"payment:sum:amount:1.00"
"payment:sum:currency:975"
"payment:type:IPGPurchase"
```

Join all strings into a single string by using semicolon (;) as a delimiter:

"carddata:cardholdername:Test;carddata:expmonth:06;carddata:expyear:25;carddata:pan:532610***0004;carddata:type:MasterCard;customer:email:payment.test@test.com;customer:identifier:SZ-1868;customer:ipaddress:127.0.0.1;operation:code:0;operation:date:2025-05-12T18:04:21+03:00;operation:eci:05;operation:message:Success;operation:provider:approval:CA0DD8;operation:provider:date:2025-05-12T18:04:19+03:00;operation:provider:respcode:00;operation:provider:trn:20250512150419228574;operation:status:success;operation:storecard:cardtoken:40B1B011C4A21EA65A8AA06E9D767ECE348ADB2E2D4E4E6C3A0536E452619059;operation:sum:amount:1.00;operation:sum:currency:975;operation:type:authorization;payment:context:CardPay;payment:date:2025-05-12T18:04:21+03:00;payment:mid:000000000000112;payment:orderid:8A6E3069-E3B2-4EB7-976F-5A6319004F26;payment:status:success;payment:sum:amount:1.00;payment:sum:currency:975;payment:type:IPGPurchase"

Get public key:

PHP

```
$publicKey = openssl_get_publickey (publicKeyString);
```

C#

```
var publicCert = File.ReadAllText(publicKeyString);
var verifyKey = RSA.Create();
verifyKey.ImportFromPem(Cert.ToCharArray());
```

Verify signature:

PHP

```
$result = openssl_verify($dataToVerify, base64_decode($signature), $publicKey);
If ($result)
{
        //success
}
Else
{
        //failed
}
```

C#

```
var sha = SHA256.Create();
var result = verifyKey.VerifyHash(sha.ComputeHash(Encoding.UTF8.GetBytes(dataToVerify)),
Convert.FromBase64String(signature), HashAlgorithmName.SHA256, RSASignaturePadding.Pkcs1);
If (result)
{
        //success
}
Else
{
        //failed
}
```

Callbacks

Overview

A callback is a system message sent from the **IPG API** payment platform to the merchant's web service. It contains information about a specific event in the payment platform that usually takes place in the context of processing a payment or storing customer payment data.

Handling callbacks

A callback is an HTTP POST message sent to the URL provided by the merchant. The procedure of responding to each callback on the web service side consists of the following steps:

1. **Accept and verify the callback.**

Callbacks should be accepted only if they have been sent from the payment platform's IP addresses provided by the iCard technical support specialists.

The callback sender and data integrity should be validated by way of verifying the signature included in every callback.

2. **Confirm the callback receipt**.

For confirming the receipt of callbacks, synchronous HTTP responses should be sent to the payment platform with the corresponding response codes. If no errors have been detected upon the receipt, the response code should be 200 OK. In other cases, the response code should correspond to the error type: for example, use HTTP 400 Bad Request if a parameter string could not be converted into an array or HTTP 500 Internal Server Error if the callback has been received at an incorrect URL of the web service.

If the 200 OK response has not been received from the web service, the callback will be sent again regardless of the error type.

3. **Perform the required actions.**

Upon the receipt of prescriptive callbacks, the actions that are stated in these callbacks as required should be performed; and upon the receipt of informational callbacks, the actions that correspond to the aspects of the web service operation should be performed (for example, customer notification).

Resending callbacks

If the information about a callback receipt error has been communicated to the payment platform or the callback receipt has not been confirmed, this callback is sent again.

Generally, resending callbacks is carried out as follows:

1. 20 attempts at interval of 60 seconds.
2. 8 attempts at interval of 300 seconds.
3. 6 attempts at interval of 600 seconds.
4. 6 attempts at interval of 1200 seconds.
5. 6 attempts at interval of 3600 seconds.

6. 7 attempts at interval of 7200 seconds until a total of 53 attempts is reached. After that, callbacks triggered by the given event are no longer sent

In general, the time span for automatic callback delivery attempts does not exceed 1 day.

Troubleshooting

There can be cases when callbacks are not received at the specified URLs for various reasons. These situations and the ways to resolve them can be divided into the following groups:

- *There are no callbacks triggered by any of the events.*

This can happen because certain requests and events have not been initiated in the platform, or issues with communication channels have been detected, or the web service URLs are invalid.

In such cases, it is recommended to proceed as follows:

1. Ensure that the correct requests (i.e. they do not disable callbacks) were sent from the web service and accepted in the platform. It may be necessary to send a test request, for example, to generate a payment card token.
2. If requests are accepted in the platform, but callbacks are still not received, check if the URLs for delivering callbacks are correct.
3. If the previous steps did not help, contact the iCard technical integration support specialists to cs.integration@icard.com or cs.support@icard.com .

Common use cases

Declined 3DS – frictionless flow


```
{
        "CardData": {
                "Pan": "400609***0007",
                "Type": "VISA",
                "CardholderName": "Test",
                "ExpMonth": "06",
                "ExpYear": "26"
        },
        "Customer": {
                "Email": "test@test.com",
                "Phone": "+35988123456",
                "Identifier": "SZ-1868",
                "IPAddress": "127.0.0.1"
        },
        "Payment": {
                "OrderId": "3AC309F6-5184-4721-B3E1-8CA08070235B",
                "MID": "000000000000113",
                "Date": "2025-05-14T10:45:30+03:00",
                "Type": "IPGPurchase",
                "Context": "",
                "Status": "declined",
                "Sum": {
                        "Amount": "20.00",
                        "Currency": 975
                },
                "Interface": "modal",
                "Description": "note"
        },
```

```
		"Operation" {
			"Type": "3ds_authentication",
			"Status": "declined",
			"Date": "2025-05-14T10:45:30+03:00",
			"Code": 3008,
			"Message": "No Card record",
			"Sum": {
				"Amount": "20.00",
				"Currency": 975
			}
		},
		"Signature": "CX7IqNnJGHej9nzd ... PoZXJHkoTpBMGMuybblrG9jU="
	}
```

Failed merchant validation:


```
{
	"Payment": {
		"OrderId": "1A3BBFE5-78B4-46C6-900A-853006365A08",
		"MID": "000000000000112",
		"Date": "2025-05-14T10:40:11+03:00",
		"Type": "IPGPurchase",
		"Context": "",
		"Status": "declined",
		"Sum": {
			"Amount": "1.00",
			"Currency": 978
		},
		"Interface": "redirect",
		"Description": "notee"
	},
	"Operation": {
		"Type": "merchant_validation",
		"Status": "declined",
		"Date": "2025-05-14T10:40:11+03:00",
		"Code": 9005,
		"Message": "Invalid parameter"
	},
	"Errors": [
		{
			"Code": 9033,
			"Description": "Invalid banner index",
			"Field": "BannerIndex",
			"Message": "Invalid integer for banner index value"
		}
	],
	"Signature": "RmGl2DCsQba8 .... 3eqNbjXBVSUdb3Lxx9zCWu/M="
}
```


Declined 3DS – challenge flow


```
{
	"CardData": {
		"Pan": "400609***0007",
		"Type": "VISA",
		"CardholderName": "Test",
		"ExpMonth": "06",
		"ExpYear": "26"
	},
	"Customer": {
		"Email": "test@test.com",
		"Phone": "+35988123456",
		"Identifier": "SZ-1868",
		"IPAddress": "127.0.0.1"
	},
	"Payment": {
		"OrderId": "99D72F74-0A57-43B7-A948-2AB4E1145662",
		"MID": "000000000000113",
		"Date": "2025-05-14T10:53:20+03:00",
		"Type": "IPGPurchase",
		"Context": "",
		"Status": "declined",
		"Sum": {
			"Amount": "201.00",
			"Currency": 975
		},
```

```
                        "Interface": "modal",
                        "Description": "note"
                },
                "Operation" {
                        "Type": "3ds_challenge",
                        "Status": "declined",
                        "Date": "2025-05-14T10:53:20+03:00",
                        "Code": 3001,
                        "Message": "Card authentication failed",
                        "Sum": {
                                "Amount": "201.00",
                                "Currency": 975
                        }
                },
                "Signature": "CX7IqNnJGHej9nzd ... PoZXJHkoTpBMGMuybblrG9jU="
        }
```

Declined payment:


```
{
	"CardData": {
		"Pan": "539260***6203",
		"Type": "MasterCard",
		"CardholderName": "Test",
		"ExpMonth": "06",
		"ExpYear": "25"
	},
	"Customer": {
		"Email": "test@test.com",
		"Phone": "+35988123456",
		"Identifier": "SZ-1868",
		"IPAddress": "127.0.0.1"
	},
	"Payment": {
		"OrderId": "E34C4058-D375-4B19-A2FF-8FA0EAC4639E",
		"MID": "000000000000113",
		"Date": "2025-05-14T10:56:04+03:00",
		"Type": "IPGPurchase",
		"Context": "",
		"Status": "declined",
		"Sum": {
			"Amount": "20.00",
			"Currency": 975
		},
		"Interface": "modal",
		"Description": "note"
	},
	"Operation" {
		"Type": "authorization",
		"Status": "declined",
		"Date": "2025-05-14T10:56:04+03:00",
		"Code": 1005,
		"Message": "Refer to card Issuer",
		"Sum": {
			"Amount": "20.00",
			"Currency": 975
		},
		"Provider": {
			"Trn": "20250514075602257447",
			"Date": "2025-05-14T10:56:02+03:00",
			"RespCode": "05"
		}
	},
	"Signature": "CX7IqNnJGHej9nzd ... PoZXJHkoTpBMGMuybblrG9jU="
}
```

Success payment:


```
{
	"CardData": {
		"Pan": "532610***0004",
		"Type": "MasterCard",
		"CardholderName": "Test",
		"ExpMonth": "06",
		"ExpYear": "25"
	},
	"Customer": {
		"Email": "test@test.com",
		"Phone": "+35988123456",
		"Identifier": "SZ-1868",
		"IPAddress": "127.0.0.1"
	},
	"Payment": {
		"OrderId": "4C498AA8-DA12-4D5D-94C3-257A29415DAF",
		"MID": "000000000000113",
		"Date": "2025-05-14T10:51:43+03:00",
		"Type": "IPGPurchase",
		"Context": "",
		"Status": "success",
		"Sum": {
			"Amount": "20.00",
			"Currency": 975
		},
		"Interface": "modal",
		"Description": "note"
	},
	"Operation" {
		"Type": "authorization",
		"Status": "success",
		"Date": "2025-05-14T10:51:43+03:00",
		"Code": 0,
		"Message": "Success",
		"Sum": {
			"Amount": "20.00",
			"Currency": 975
		},
		"Provider": {
			"Trn": "20250514075143257397",
			"Date": "2025-05-14T10:51:41+03:00",
			"RespCode": "00",
			"Approval": "SWCSIM"
		},
		"Eci": "05"
	},
	"Signature": "CX7IqNnJGHej9nzd ... PoZXJHkoTpBMGMuybblrG9jU="
}
```

Callback parameters

Object Payment

This object contains essential payment details**. The object is mandatory**.

Sub Object	Property	Typical value	Type	Requirement status	Description
	OrderId	78A9F22B-FE59-4170-8EA6-2BDE716E07D5	String	Optional	Placeholder for the merchant.
	MID	000000000000123	AN(15)	Optional	Identifier of the virtual terminal used for the purchase.
	Date	2025-01-30T17:57:46+02:00	Date ISO 8601	Mandatory	Date of last status update. Example - 2025-01-30T12:39:13+02:00
	Type	IPGPurchase	String	Optional	Name of the method requested for execution from IPG.
	Context	CardPay	String	Optional	Context of the payment process with Card - CardPay, GooglePay, ApplePay, StoreCardPay and iCard reserved other future payment context.
	Status	success	String	Mandatory	success, error, declined
	Interface	redirect	String	Mandatory	embedded_checkout, redirect, modal, backend_request
Sum				Optional	
	Amount	1.00	Double (8,2)	Mandatory	Echo from method input data
	Currency	978	N (3)	Mandatory	Echo from method input data

Object CardData

This object contains customer's card data. **The object is optional.**

Object CardData properties

Property	Typical value	Type	Requirement status	Description
Pan	532610***0004	String	Mandatory	First 6 and last 4 digits of PAN.
Type	MasterCard	String	Mandatory	Mastercard, VISA.
CardholderName	First and Last Name	String	Mandatory	Customer's name
ExpMonth	06	String	Mandatory	Expire month of the card
ExpYear	25	String	Mandatory	Expire year of the card
CardToken	D747458899D….FC43D5	String (64)	Mandatory	Token of the card used for subsequent payments. Token of the card in case client choose checkbox - Save card during IPGPurchase.
IssCountry	ISR	String	Mandatory	Issuing Country of the card.
IssRegion	D	String	Mandatory	Issuing Region of the card.
AcqScheme	M	String	Mandatory	Acquiring scheme used for the transaction.
Brand	M	String	Mandatory	Brand of the card used.
ProductID	MCC	String	Mandatory	Product ID of the card used.
ProductClass	MCC	String	Mandatory	Product Class of the card used.
ProductClassName	Consumer	String	Mandatory	Product Class Name of the card used.
Qualifier	MasterCard	String	Mandatory	Qualifier of the card used.

Object Customer

This object contains customer's card data. **The object is optional.**

Object Customer properties

Property	Typical value	Type	Requirement status	Description
Email	test@test.com	String	Optional	Echo from method input data
Phone	+359881252525	String	Optional	Echo from method input data
Identifier	SZ-1868	String	Optional	Echo from method input data
IPAddress	127.0.0.1	String	Optional	Dotted-decimal string, that holds the customer IP address as reported at merchant web shop.
FirstName	First Name	String	Optional	Echo from method input data
LastName	Last Name	String	Optional	Echo from method input data

Object Operation

This object contains information about the operations related to the payment. **The object is optional.**

Object Operation properties

Sub Object	Property	Typical value	Type	Requirement status	Description
	Type	authorization	String	Optional	authorization, merchant_validation, customer_validation, 3ds_authentication, 3ds_challenge, store_card
	Status	success	String	Mandatory	success, error, declined
	Date	2025-01-30T17:57:45+02:00	Date ISO 8601	Mandatory	Date of last status update.
	Code	0	String	Mandatory	Refer to document IPG Additions Status Codes
	Message	Success	String	Mandatory	Refer to document IPG Additions Status Codes
	Eci	04	String	Optional	Refer to Electronic Commerce Indicator (Eci) Field values
Sum					
	Amount	10.50	Double(8,2)	Mandatory	Echo from method input data
	Currency	978	N (3)	Mandatory	Echo from method input data
Provider				Optional	
	Trn	20250130155745249280	String	Mandatory	Payment identifier on a provider or a payment system side.
	Date	2025-01-30T17:57:45+02:00	Date ISO 8601	Mandatory	Date and time of the payment processing completion on a provider or a payment system side
	RespCode	00		Mandatory	Response code returned to the transaction from Card Scheme.
	Approval	F1DA09	String	Optional	Approval Code returned to transaction which are approved with RC:00 from Issuer.
StoreCard				Optional	
	CardToken	D747458899D....FC43D5	String (64)	Mandatory	Token of the card used for subsequent payments. Token of the card in case client choose checkbox - Save card during IPGPurchase.

Electronic Commerce Indicator (Eci) Field values:

Payment System specific value provided by the ACS or DS to indicate the results of the attempt to authenticate the Cardholder.

Property	Typical value	Type	Description
Eci	**For MasterCard** 00 – Merchant not participating in 3D program or card enrollment service is unavailable 01 – Attempted card 02 – full 3D authentication 06 – full 3D authentication in case of SCA **For VISA** 05 – full 3D authentication 06 – Attempted card or not participating but the merchant is certified for 3D 07 – Merchant not participating in 3D program or card enrollment service is unavailable **For AMEX** 05 – Authenticated with AEVV 06 – Attempted with AEVV 07 – Not Authenticated 08 – This is a phone order transaction	String (2)	Electronic commerce indicator. Shows the enrollment of the cardholder in MasterCard 3D Secure, Verified by Visa or Amex Safekey programs.

Array Errors

Array of error messages received during request execution. Optional

Object ErrorItem

Object with information about the error that occurred during request execution. Required

Object ErrorItem properties

Property	Typical value	Type	Requirement status	Description
Code	9033	Integer	Optional	Error code
Description	Invalid banner index	String	Optional	Error reason details.
Field	BannerIndex	String	Optional	Name of the parameter that was erroneously specified (if such parameter is identified).
Message	Invalid integer for banner index value	String	Optional	Error code description.

IPG Implementations with methods

Type of IImplementation	API function call
Redirect Checkout	IPGPurchase
	IPG3DSPurchaseWithStoredCard
Embedded checkout	IPGEmbeddedPayment with PaymentType IPGPurchase
	IPGEmbeddedPayment with PaymentType IPG3DSPurchaseWithStoredCard
Modal Implementation	IPGPaymentToken with ModalType IPGPurchase
	IPGPaymentToken with ModalType IPG3DSPurchaseWithStoredCard
Google Pay Implementation	Google payment method on merchant page
	IPGPurchase
Apple Pay JS SDK with redirect Implementation	Apple payment method on merchant page
	IPGPurchase
Apple Pay only JS SDK Implementation	Apple payment method on merchant page
	IPGTokenProviderSession
	IPGTokenizedCardPurchase
Backend Implementation	IPGOCT
	IPGGetTxnStatus
	IPGReversal

Response standard properties

Upon an HTTP request, the responding party MUST return an HTTP response with a status code of 200 OK.

Every other response should be treated as communication error, call error, server error or system malfunctions.

Redirect Checkout Implementation

Purchase with payment card (API call: IPGPurchase)

Purpose

This method initiates the beginning of the payment process for a cardholder. The cardholder is placed on a Checkout page with **GPay** button.

IPG will check:

- Valid MID.
- Valid currency with regards to the MID.
- Properties MID in combination with OrderID gives a unique identifier for the request of a partner. IPG will reject duplicated transmission.

Method properties

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGPurchase	String	Mandatory	Name of the method requested for execution from IPG.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature.
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	string	Mandatory	Version of protocol used for transition.
Language	EN	A(2)	Mandatory	ISO 2-character code for the desired language on the payment page. If IPG cannot fulfill the requested language, it will set the English language as default. Currently supporting EN, FR, DE, BG, ES, RO, EL, IT, PL.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
BannerIndex	1	Int	Mandatory	Index specified in IPG for every banner provided by the Merchant. The Merchant may choose to select a proper banner for every payment. The banner is displayed on the payment page.
PostResultAction	Redirect	String	Conditional	Values of the parameter: **Redirect** – redirect to URL_OK and URL_Cancel **CloseWindow** – close the window after the transaction and merchant have control of next actions If there is no value in this parameter, iCard will show iCard page for the result of transaction and after that it will redirect to merchant site.
MID	000000000000123	AN(15)	Mandatory	Identifier of the virtual terminal used for the purchase.

MIDName	Merchant Web Shop	String	Mandatory	User friendly name of the merchant web shop. The cardholder will see this name on some notices in the payment page.
Amount	23.45	Double (8,2)	Mandatory	The amount of the payment requested.
Currency	978	N(3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
CustomerIP	127.0.0.1	String	Mandatory	Customer IP address in IPv4 or IPv6 format
OrderID	60EC4A03-0AC1-44E5-872B-08FC5AAC9FDB	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.
CustomerIdentifier	1234	String	Recommended	Credentials of the customer at merchant platform (ID, phone number or names).
Email	customer@mywebsite.com	String	Mandatory	This is the cardholder's email.
URL_OK	http://site.ext/paymentOK	String	Conditional	The page where the cardholder should be redirected on successful payment. Do not required for PostResultAction = CloseWindow.
URL_Cancel	http://site.ext/paymentNOK	String	Conditional	The page where the cardholder should be redirected when the transaction is unsuccessful. Do not required for PostResultAction = CloseWindow.
URL_Notify	http://site.ext/paymentNotify	String	Mandatory	Address supplied by the partner, where iPG API will send the callbacks.
Note	Note	String	Optional	Text associated with the purchase.
MobileNumber	+359811222111	String	Mandatory	Cardholder's mobile number. Format of the phone number is always with Country code filled in the beginning. Every other format will be rejected.
BillAddrCountry	100	String (3)	Recommended	ISO 3166-1 numeric three-digit country code of the customer's billing country.
BillAddrCity	Sofia	String (50)	Recommended	Customer's billing city.
BillAddrPostCode	1421	String (16)	Recommended	Customer's billing ZIP code.
BillAddrState		String (3)	Conditional	Country subdivision code defined in ISO 3166-2. Valid only for USA.
BillAddrLine1	128 Dondukov Blvd	String (50)	Recommended	Billing Address Line 1. Encoded in BASE 64 format. The specified length refers to the address before encoding.
BillAddrLine2		String (50)	Optional	Billing Address Line 2. Encoded in BASE 64 format. The specified length refers to the address before encoding.
BillAddrLine3		String (50)	Optional	Billing Address Line 3. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrCountry	100	String (3)	Optional	ISO 3166-1 numeric three-digit country code of customer's shipping country.
ShipAddrCity		String (50)	Optional	Customer's shipping city.
ShipAddrPostCode		String (16)	Optional	Customer's shipping ZIP code.

ShipAddrState		String (3)	Optional	Country subdivision code defined in ISO 3166-2. Valid only for USA
ShipAddrLine1		String (50)	Optional	Shipping Address Line 1. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrLine2		String (50)	Optional	Shipping Address Line 2. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrLine3		String (50)	Optional	Shipping Address Line 3. Encoded in BASE 64 format. The specified length refers to the address before encoding.
Signature	Byte[]	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Processing Purchase with stored card with 3DS verification (API call: IPG3DSPurchaseWithStoredCard)

Purpose

This method is used by IPG to allow processing of purchase with stored card with 3DS. It requires opening of new tab/window and redirecting of cardholder to the page issuer ACS where 3DS check will be executed.

IPG will check:

- Valid MID.
- Valid currency with regards to the MID.

Method properties

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPG3DSPurchaseWithStoredCard	String	Mandatory	Name of the method requested for execution from IPG.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	String	Mandatory	Version of protocol used for transition.
Language	EN	A(2)	Mandatory	Conditional only for Front end methods ISO 2-character code for the desired language on the payment page. If IPG cannot fulfill the requested language, it will set the English language as default. Currently supporting EN, FR, DE, BG, ES, RO, EL, IT, PL.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD
BannerIndex	1	Int	Mandatory	Index specified in IPG for every banner provided by the Merchant. The Merchant may choose to select a proper banner for every payment. The banner is displayed on the payment page.
PostResultAction	Redirect	String	Conditional	Values of the parameter: **Redirect** – redirect to URL_OK and URL_Cancel **CloseWindow** – close the window after the transaction and merchant have control of next actions If there is no value in this parameter, iCard will show iCard page for the result of transaction and after that it will redirect to merchant site.
MID	000000000123	AN(15)	Mandatory	Identifier of the virtual terminal used for the purchase.
MID name	Merchant Web Shop	String	Mandatory	User friendly name of the merchant web shop. The cardholder will see this name on some notices in the payment page.

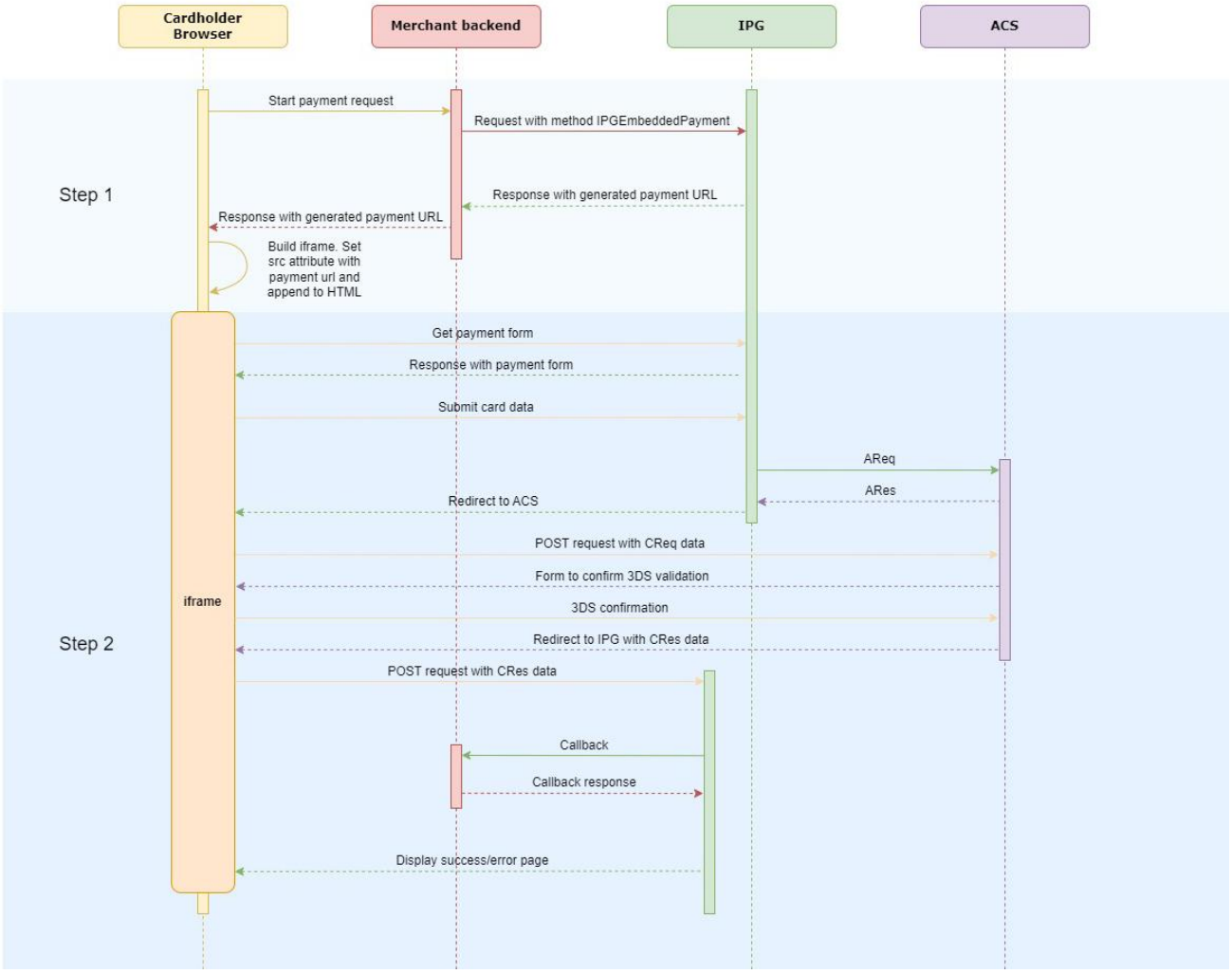
Amount	23.45	Double (8,2)	Mandatory	The amount of the payment requested.
Currency	978	N (3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
OrderID	20210916999999	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.
CardToken	40B1B011C4A21EA65A8AA06E9D767ECE348ADB2E2D4E4E6C3A0536E452619059	String	Mandatory	Token of the card. Received in Notify callback in case customer choose check-box Save card during IPGPurchase.
VerifyCVC	1	N(1)	Optional	If sent with value = 1, the customer will need to confirm the card's CVC, before proceeding with the payment.
URL_OK	http://site.ext/paymentOK	String	Conditional	The page where the cardholder should be redirected on successful payment. Do not required for PostResultAction = CloseWindow.
URL_Cancel	http://site.ext/paymentNOK	String	Conditional	The page where the cardholder should be redirected when the transaction is unsuccessful. Do not required for PostResultAction = CloseWindow.
URL_Notify	http://site.ext/paymentNotify	String	Mandatory	Address supplied by the partner, where iPG API will send the callbacks.
CustomerIdentifier	123456789	String	Recommended	Credentials of the customer at merchant platform (ID, phone number or names).
E-mail	name@website.com	String	Mandatory	This is the cardholder's email.
MobileNumber	+359811222111	String	Mandatory	Cardholder's mobile number. Format of the phone number is always with Country code filled in the beginning. Every other format will be rejected.
BillAddrCountry	100	String (3)	Recommended	ISO 3166-1 numeric three-digit country code of the customer's billing country.
BillAddrCity	Sofia	String (50)	Recommended	Customer's billing city.
BillAddrPostCode	1421	String (16)	Recommended	Customer's billing ZIP code.
BillAddrState		String (3)	Conditional	Country subdivision code defined in ISO 3166-2 . Valid only for USA.
BillAddrLine1	128 Dondukov Blvd	String (50)	Recommended	Billing Address Line 1. Encoded in BASE 64 format. The specified length refers to the address before encoding.
BillAddrLine2		String (50)	Optional	Billing Address Line 2. Encoded in BASE 64 format. The specified length refers to the address before encoding.
BillAddrLine3		String (50)	Optional	Billing Address Line 3. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrCountry	100	String (3)	Optional	ISO 3166-1 numeric three-digit country code of customer's shipping country.
ShipAddrCity		String (50)	Optional	Customer's shipping city.
ShipAddrPostCode		String (16)	Optional	Customer's shipping ZIP code.

ShipAddrState		String (3)	Optional	Country subdivision code defined in ISO 3166-2. Valid only for USA
ShipAddrLine1		String (50)	Optional	Shipping Address Line 1. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrLine2		String (50)	Optional	Shipping Address Line 2. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrLine3		String (50)	Optional	Shipping Address Line 3. Encoded in BASE 64 format. The specified length refers to the address before encoding.
Signature	Byte[]	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash

Embedded checkout Implementation

IPG Embedded Payment is a piece of code that enables customers to make payments without leaving the merchant Web page. It works like a mini-application that is used to securely collect payment data and process transaction. Embedded Payment allows processing without redirects in order to improve the user experience, increase the level of trust and comfort of the end customer. In order to get an embedded checkout in the merchant's page, the merchant needs to provide parameters related to the visualization of the payment page in advance. The set of parameters in CSS file is provided at the beginning of the implementation process from support engineers.

Process flow of transaction with Embedded checkout



Create payment interface

In order to create a payment interface, Merchant needs to append iframe element to his own page. The data SRC attribute of iframe must be set with URL provided through API call IPGEmbeddedPayment request on **Step 1**.
After successful payment, Merchant system will receive asynchronous request on URL_Notify with details.

Payment URL Request (API Call: IPGEmbeddedPayment)

Purpose

Payment URL Request is a back-end synchronous request. Merchant's system sends all required information about the payment transaction. In response, it is received a **URL**, which must be used in **Step 2.**

IPG will check:

- Valid MID.
- Valid currency with regards to the MID.
- Properties MID in combination with OrderID gives a unique identifier for the request of a partner. IPG will reject duplicated transmission.

IPGEmbeddedPayment can be implemented with two types of payments:

- **IPGPurchase** – this payment method allows the cardholder to make Payment with card and option for store card for future payments
- **IPG3DSPurchaseWithStoredCard** - This method is used by IPG to allow processing of purchase with stored card with 3DS.

Every time a content is loaded into the iframe, POST messages are sent to parent – web site of the merchant. The structure and properties of the messages are the same as Callbacks and the only difference is that there is no Signature.

Method request properties IPGEmbeddedPayment with PaymentType IPGPurchase

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGEmbeddedPayment	String	Mandatory	Name of the method requested for execution from IPG.
PaymentType	IPGPurchase	String	Mandatory	Type of payment method
Theme	Themename	String	Mandatory	The data for Theme will be provided from iCard after filled CSS file with parameters for visualization is received from Merchant.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature.
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	string	Mandatory	Version of protocol used for transition.
Language	EN	A(2)	Mandatory	ISO 2-character code for the desired language on the payment page. If IPG cannot fulfill the requested language, it will set the English language as default. Currently supporting EN, FR, DE, BG, ES, RO, EL, IT, PL.
OutputFormat	json	String	Optional	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.
Originator	100	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
MID	000000000000123	AN (15)	Mandatory	Identifier of the virtual terminal used for the purchase.
Amount	23.45	Double (8,2)	Mandatory	The amount of the payment requested.
Currency	978	N (3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
CustomerIP	127.0.0.1	String	Mandatory	Dotted-decimal string, that holds the customer IP address as reported at merchant web shop.
OrderID	47A11480-B3AA-416B-83AF-273A27EA78AA	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.
URL_Notify	http://site.ext/paymentNotify	String	Mandatory	Address supplied by the partner, where iPG API will send the callbacks.
CustomerIdentifier	1234	String	Recommended	Credentials of the customer at merchant checkout page (ID, phone number or names). Will be returned in IPGPurchaseNotify if sent in the POST request from the merchant

Email	customer@mywebsite.com	String	Mandatory	This is the cardholder's email.
MobileNumber	+359811222111	String	Mandatory	Cardholder's mobile number. Format of the phone number is always with Country code filled in the beginning. Every other format will be rejected.
BillAddrCountry	100	String (3)	Recommended	ISO 3166-1 numeric three-digit country code of the customer's billing country. This field is always BASE 64 encoded.
BillAddrCity	Sofia	String (50)	Recommended	Customer's billing city.
BillAddrPostCode	1421	String (16)	Recommended	Customer's billing ZIP code.
BillAddrState		String (3)	Conditional	USA only
BillAddrLine1	128 Dondukov Blvd	String (50)	Recommended	Billing Address Line 1. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
BillAddrLine2		String (50)	Optional	Billing Address Line 2. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
BillAddrLine3		String (50)	Optional	Billing Address Line 3. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
ShipAddrCountry	100	String (3)	Optional	ISO 3166-1 numeric three-digit country code of customer's shipping country.
ShipAddrCity	Sofia	String (50)	Optional	Customer's shipping city.
ShipAddrPostCode	1421	String (16)	Optional	Customer's shipping ZIP code.
ShipAddrState		String (3)	Optional	USA only
ShipAddrLine1	128 Dondukov Blvd	String (50)	Optional	Shipping Address Line 1. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
ShipAddrLine2		String (50)	Optional	Shipping Address Line 2. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
ShipAddrLine3		String (50)	Optional	Shipping Address Line 3. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
Signature	uIkMPIYhgvCBzCnCymLxEyeI6c0mKd.....7DEhXOaUKakY=	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Method response properties IPGEmbeddedPayment with PaymentType IPGPurchase

Property	Typical value	Type	Description
IPGmethod	IPGEmbeddedPayment	String	Name of the method
OrderID	47A11480-B3AA-416B-83AF-273A27EA78AA	String	Echo from IPGEmbeddedPayment.
Status	0	String	Request status. Refer to document IPG Additions Status Codes.
StatusMsg	Success	String	Status message. Refer to document IPG Additions Status Codes.
URL	https://dev-ipg…	String	URL for iframe
Signature	uIkMPIYhgvCBzCnCymLxEyel6c0mKd.....7DEhXOaUKakY=	BASE64	Signed HASH for all properties in the command. Signature is ALWAYS THE LAST PARAMETER IN THE POST, as it is not used to calculate the hash.

Method properties IPGEmbeddedPayment with PaymentType IPG3DSPurchaseWithStoredCard

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGEmbeddedPayment	String	Mandatory	Name of the method requested for execution from IPG.
Theme	Themename	String	Mandatory	The data for Theme will be provided from iCard after filled CSS file with parameters for visualization is received from Merchant.
PaymentType	IPG3DSPurchaseWithStoredCard	String	Mandatory	Type of payment method which will be run in modal. Possible values are: - IPGPurchase - IPG3DSPurchaseWithStoredCard
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature.
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant.
IPGVersion	4.5	String	Mandatory	Version of protocol used for transition.
Language	EN	A(2)	Mandatory	ISO 2-character code for the desired language on the payment page. If IPG cannot fulfill the requested language, it will set the English language as default. Currently supporting EN, FR, DE, BG, ES, RO, EL, IT, PL.
OutputFormat	json	String	Optional	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.
Originator	100	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
MID	000000000000123	AN(15)	Mandatory	Identifier of the virtual terminal used for the purchase.
Amount	23.45	Double(8,2)	Mandatory	The amount of the payment requested.
Currency	978	N(3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
CustomerIP	127.0.0.1	String	Mandatory	Dotted-decimal string, that holds the customer IP address as reported at merchant web shop.
OrderID	B0082D58-84B1-4143-8C9E-3F89B57AA232	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.
URL_Notify	http://site.ext/paymentNotify	String	Mandatory	Address supplied by the partner, where iPG API will send the callbacks.
CustomerIdentifier	1234	String	Recommended	Credentials of the customer at merchant checkout page (ID, phone number or names). Will be returned in IPGPurchaseNotify if sent in the POST request from the merchant
Email	customer@mywebsite.com	String	Mandatory	This is the cardholder’s email.
CardToken	D747458899D….FC43D5	String	Mandatory	Token of the card. Required for embedded payment method IPG3DSPurchaseWithStoredCard.
VerifyCVC	1	N(1)	Optional	If sent with value = 1, the customer will need to confirm the card’s CVC, before proceeding with the payment. Valid for embedded payment method IPG3DSPurchaseWithStoredCard.

MobileNumber	+359811222111	String	Mandatory	Cardholder's mobile number. Format of the phone number is always with Country code filled in the beginning. Every other format will be rejected.
BillAddrCountry	100	String (3)	Recommended	ISO 3166-1 numeric three-digit country code of the customer's billing country.
BillAddrCity	Sofia	String (50)	Recommended	Customer's billing city.
BillAddrPostCode	1421	String (16)	Recommended	Customer's billing ZIP code.
BillAddrState		String (3)	Conditional	USA only
BillAddrLine1	128 Dondukov Blvd	String (50)	Recommended	Billing Address Line 1. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
BillAddrLine2		String (50)	Optional	Billing Address Line 2. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
BillAddrLine3		String (50)	Optional	Billing Address Line 3. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
ShipAddrCountry	100	String (3)	Optional	ISO 3166-1 numeric three-digit country code of customer's shipping country.
ShipAddrCity	Sofia	String (50)	Optional	Customer's shipping city.
ShipAddrPostCode	1421	String (16)	Optional	Customer's shipping ZIP code.
ShipAddrState		String (3)	Optional	Country subdivision code defined in ISO 3166-2
ShipAddrLine1	128 Dondukov Blvd	String (50)	Optional	Shipping Address Line 1. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
ShipAddrLine2		String (50)	Optional	Shipping Address Line 2. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
ShipAddrLine3		String (50)	Optional	Shipping Address Line 3. This field is always BASE 64 encoded. The specified length refers to the address before encoding.
Signature	uIkMPIYhgE.....7DEhXOaUKakY=	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

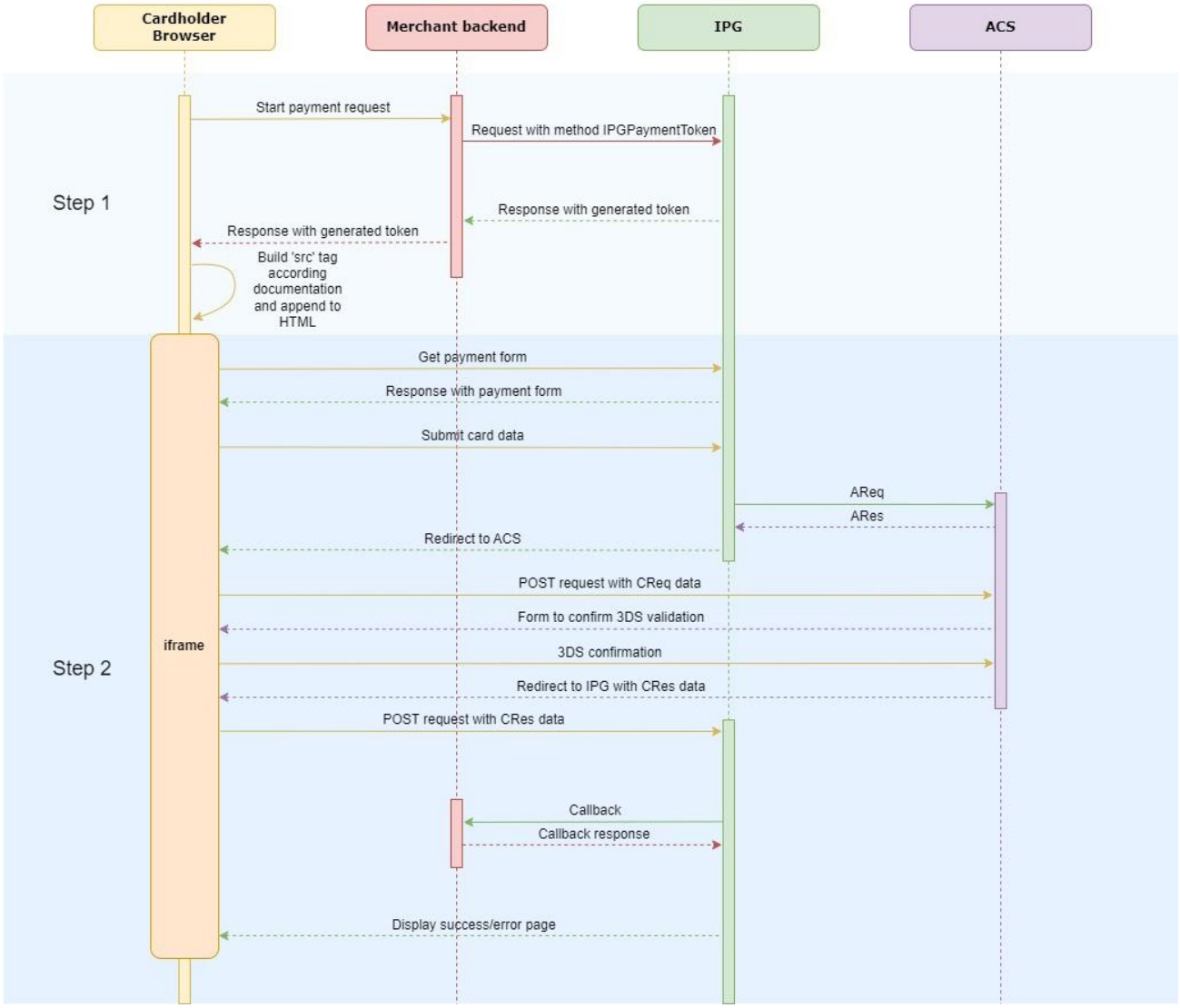
Method response properties IPGEmbeddedPayment with PaymentType IPG3DSPurchaseWithStoredCard

Property	Typical value	Type	Description
IPGmethod	IPGEmbeddedPayment	String	Name of the method
OrderID	47A11480-B3AA-416B-83AF-273A27EA78AA	String	Echo from IPGEmbeddedPayment.
Status	0	String	Request status. Refer to document IPG Additions Status Codes.
StatusMsg	Success	String	Status message. Refer to document IPG Additions Status Codes.
URL	https://dev-ipg…	String	URL for iframe
Signature	uIkMPIYhgE.....7DEhXOaUKakY=	BASE64	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Modal Implementation

IPG Payment modal is piece of code that provides customers to making payments without leaving Web page of store. It works like a mini – application that is used to securely collect payment data and process transaction. Payment modal allows processing without redirects in order to improve the user experience and to increase the level of trust and comforts of end customers.

Process flow of transaction with Modal:



Payment Token Request (API Call: IPGPaymentToken)

Purpose

Payment Token Request is a back-end synchronous request. Merchant' system sends all required information about the payment transaction. In response, it is received a **token**, which must be used in the next step 2.

Method properties IPGPaymentToken for Modal type IPGPurchase

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGPaymentToken	String	Mandatory	Name of the method requested for execution from IPG.
ModalType	IPGPurchase	String	Mandatory	Type of payment method which will be run in modal.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature.
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	string	Mandatory	Version of protocol used for transition.
Language	EN	A(2)	Mandatory	ISO 2-character code for the desired language on the payment page. If IPG cannot fulfill the requested language, it will set the English language as default. Currently supporting EN, FR, DE, BG, ES, RO, EL, IT, PL.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
OutputFormat	json	String	Mandatory	Output format of data. The property can be "xml" or "json". If it is not specified in the request, the default value is "xml".
BannerIndex	1	Int	Conditional	Index specified in IPG for every banner provided by the Merchant. The Merchant may choose to select a proper banner for every payment. The banner is displayed on the payment page.
MID	000000000000123	AN (15)	Mandatory	Identifier of the virtual terminal used for the purchase.
MIDName	Merchant Web Shop	String	Mandatory	User friendly name of the merchant web shop. The cardholder will see this name on some notices in the payment page.
Amount	23.45	Double (8,2)	Mandatory	The amount of the payment requested.
Currency	978	N (3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
CustomerIP	127.0.0.1	String	Mandatory	Customer IP address in IPv4 or IPv6 format
OrderID	60EC4A03-0AC1-44E5-872B-08FC5AAC9FDB	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.

CustomerIdentifier	1234	String	Recommended	Credentials of the customer at merchant platform (ID, phone number or names).
Email	customer@mywebsite.com	String	Mandatory	This is the cardholder's email.
URL_Notify	http://site.ext/paymentNotify	String	Mandatory	Address supplied by the partner, where iPG API will send the callbacks.
Note	Note	String	Optional	Text associated with the purchase.
MobileNumber	+359811222111	String	Mandatory	Cardholder's mobile number. Format of the phone number is always with Country code filled in the beginning. Every other format will be rejected.
BillAddrCountry	100	String (3)	Recommended	ISO 3166-1 numeric three-digit country code of the customer's billing country.
BillAddrCity	Sofia	String (50)	Recommended	Customer's billing city.
BillAddrPostCode	1421	String (16)	Recommended	Customer's billing ZIP code.
BillAddrState		String (3)	Conditional	Country subdivision code defined in ISO 3166-2. Valid only for USA.
BillAddrLine1	128 Dondukov Blvd	String (50)	Recommended	Billing Address Line 1. Encoded in BASE 64 format. The specified length refers to the address before encoding.
BillAddrLine2		String (50)	Optional	Billing Address Line 2. Encoded in BASE 64 format. The specified length refers to the address before encoding.
BillAddrLine3		String (50)	Optional	Billing Address Line 3. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrCountry	100	String (3)	Optional	ISO 3166-1 numeric three-digit country code of customer's shipping country.
ShipAddrCity		String (50)	Optional	Customer's shipping city.
ShipAddrPostCode		String (16)	Optional	Customer's shipping ZIP code.
ShipAddrState		String (3)	Optional	Country subdivision code defined in ISO 3166-2. Valid only for USA
ShipAddrLine1		String (50)	Optional	Shipping Address Line 1. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrLine2		String (50)	Optional	Shipping Address Line 2. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrLine3		String (50)	Optional	Shipping Address Line 3. Encoded in BASE 64 format. The specified length refers to the address before encoding.
Signature	Byte[]	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Method properties IPGPaymentToken for Modal type IPG3DSPurchaseWithStoredCard

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGPaymentToken	String	Mandatory	Name of the method requested for execution from IPG.
ModalType	IPG3DSPurchaseWithStoredCard	String	Mandatory	Type of payment method which will be run in modal.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	String	Mandatory	Version of protocol used for transition.
Language	EN	A(2)	Mandatory	Conditional only for Front end methods ISO 2-character code for the desired language on the payment page. If IPG cannot fulfill the requested language, it will set the English language as default. Currently supporting EN, FR, DE, BG, ES, RO, EL, IT, PL.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD
OutputFormat	json	String	Mandatory	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.
BannerIndex	1	Int	Conditional	Index specified in IPG for every banner provided by the Merchant. The Merchant may choose to select a proper banner for every payment. The banner is displayed on the payment page.
CardToken	40B1B011C4A21EA65A8AA06E9D767ECE348ADB2E2D4E4E6C3A0536E452619059	String	Mandatory	Token of the card. Token of the card is received in Notify callback in case client choose checkbox - Save card during IPGPurchase.
VerifyCVC	1	N(1)	Optional	If sent with value = 1, the customer will need to confirm the card’s CVC, before proceeding with the payment.
MID	000000000123	AN(15)	Mandatory	Identifier of the virtual terminal used for the purchase.
MID name	Merchant Web Shop	String	Mandatory	User friendly name of the merchant web shop. The cardholder will see this name on some notices in the payment page.
Amount	23.45	Double (8,2)	Mandatory	The amount of the payment requested.
Currency	978	N(3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
OrderID	60EC4A03-0AC1-44E5-872B-08FC5AAC9FDB	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.
CustomerIdentifier	123456789	String	Recommended	Credentials of the customer at merchant platform (ID, phone number or names).
E-mail	name@website.com	String	Mandatory	This is the cardholder’s email.

URL_Notify	http://site.ext/paymentNotify	String	Mandatory	Address supplied by the partner, where iPG API will send the callbacks.
Note	Note	String	Optional	Text associated with the purchase.
MobileNumber	+359811222111	String	Mandatory	Cardholder's mobile number. Format of the phone number is always with Country code filled in the beginning. Every other format will be rejected.
BillAddrCountry	100	String (3)	Recommended	ISO 3166-1 numeric three-digit country code of the customer's billing country.
BillAddrCity	Sofia	String (50)	Recommended	Customer's billing city.
BillAddrPostCode	1421	String (16)	Recommended	Customer's billing ZIP code.
BillAddrState		String (3)	Conditional	Country subdivision code defined in ISO 3166-2. Valid only for USA.
BillAddrLine1	128 Dondukov Blvd	String (50)	Recommended	Billing Address Line 1. Encoded in BASE 64 format. The specified length refers to the address before encoding.
BillAddrLine2		String (50)	Optional	Billing Address Line 2. Encoded in BASE 64 format. The specified length refers to the address before encoding.
BillAddrLine3		String (50)	Optional	Billing Address Line 3. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrCountry	100	String (3)	Optional	ISO 3166-1 numeric three-digit country code of customer's shipping country.
ShipAddrCity		String (50)	Optional	Customer's shipping city.
ShipAddrPostCode		String (16)	Optional	Customer's shipping ZIP code.
ShipAddrState		String (3)	Optional	Country subdivision code defined in ISO 3166-2. Valid only for USA
ShipAddrLine1		String (50)	Optional	Shipping Address Line 1. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrLine2		String (50)	Optional	Shipping Address Line 2. Encoded in BASE 64 format. The specified length refers to the address before encoding.
ShipAddrLine3		String (50)	Optional	Shipping Address Line 3. Encoded in BASE 64 format. The specified length refers to the address before encoding.
Signature	Byte[]	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash

Create payment form

In order to create a payment form, Merchant needs to add the following lines of HTML/JavaScript to his own page.

Wrapper

Wrapper MUST have id=" ipg". The following tag should be added in HTML code of Merchant's site:

```
<div id="ipg"></div>
```

JavaScript code

After a token value is received, the JavaScript function below has to be executed on the Merchant's website having placeholders **"_DOMAIN_ ", "_TOKEN_" and "_THEME_"** replaced respectively with:

- **"_DOMAIN_"** - Either address could be used as a domain: **"https://ipg.icard.com/"** or **"https://dev-ipg.icards.eu/sandbox/"**
- **"_THEME_"** - To be replaced with **"classic"** or **"dark",** otherwise classic will be applied automatically.
- **"_TOKEN_"** - To be replaced with token value, obtained on the step 1 as a response of API Call IPGPaymentToken request:

```
<script>
    function loadModal()
    {
        const src = _DOMAIN_ + "js/payment-modal.js?token=" + _TOKEN_ + '&theme=' + _THEME_;
        const script = document.createElement("script");
        script.src = src;
        script.id = "ipg-io-js";
        script.async = "async";
        document.querySelector('body').appendChild(script);
    }
</script>
```

After successful payment, Merchant system will receive asynchronous request on URL_Notify with details, the same one, provided through API Call IPGPaymentToken request on step 1.

Google Pay Implementation

Purpose

Google Pay is mobile payment and digital wallet service from Google, which provides secure and user-friendly solution for users to pay on the merchant website. iCard has integration with Google that provides **JS SDK with redirect** solution for the gambling businesses model which can be implemented in the merchant website.

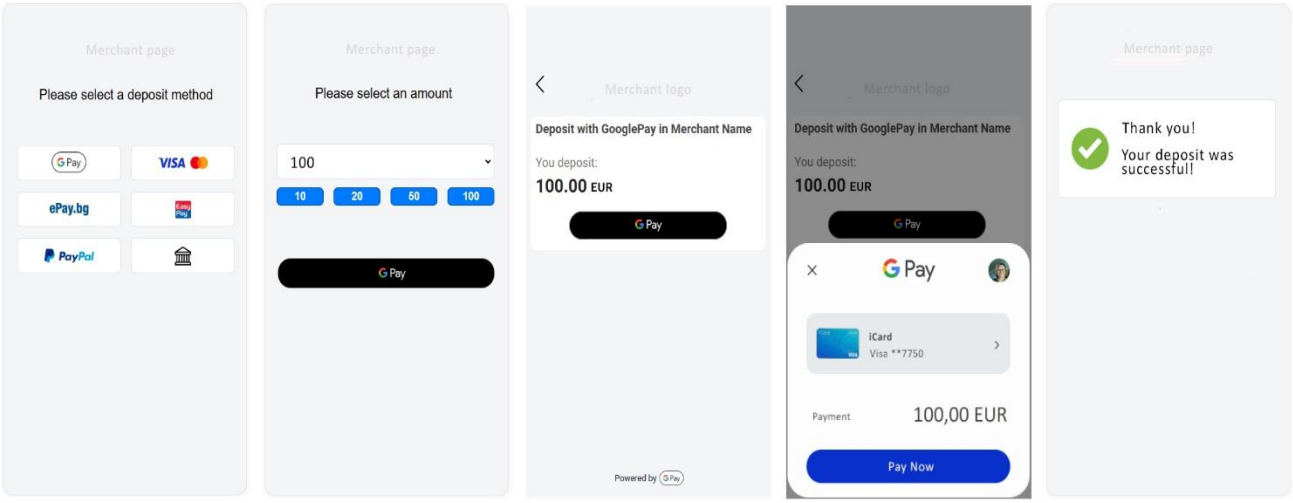
JS SDK is used for checking the availability of the GPay button on the device of the cardholder. Merchant uses redirect for payment checkout and transaction process.

Additional Details

- Google Pay is available only if the user's card is tokenized and if they use a device which allows user authentication.
- Google Pay is not available through WebView realization of mobile application.

User Flow details

- **Payment Method Selection** – When the user is browsing available deposit methods on the merchant page.
- **Pre – Deposit Screen** – Displayed when the user is about to make a deposit on the merchant page.
- **Payment Method Screen** – Shown when the user selects Google Pay as their payment method on the iCard redirect checkout.
- **Google Pay API Payment Screen** – Displays the payment information the user has saved in Google Pay.
- **Post – Deposit Screen** – Shown after the user successfully completes a deposit to the merchant page.



Google payment method on merchant page

Payment Method Selection page

In order to display **Google Pay** as a payment method, a Merchant should include the following script on their page:

HTML head section

Sandbox environment:

```
<head>
        <script src="https://dev-ipg.icards.eu/sandbox/js/icard-g-a-pay.min.js"></script>
</head>
```

Production environment:

```
<head>
        <script src="https://ipg.icard.com/js/icard-g-a-pay.min.js"></script>
</head>
```

In order to check for **GPay** availability, use the following example script:

```
<script>
  window.addEventListener('DOMContentLoaded', function () {
    const config = {
      mid: merchant id (provided by iCard), //type string
      environment: 'prod/sandbox'
    };

    const googlePay = new ICardIpgGAPay(config)

    googlePay.isGooglePayAvailable()
        .then((isReady) => {
          if (isReady) {
            //Show Google payment method btn
          }
        })
        .catch((error) => {
          console.log(error);
        })
  })
</script>
```

Purchase with Google Pay (API call: IPGPurchase)

Purpose

This method initiates the beginning of the payment process for a cardholder. The cardholder is placed on a Checkout page with **GPay** button.

IPG will check:

- Valid MID.
- Valid currency with regards to the MID.
- Properties MID in combination with OrderID gives a unique identifier for the request of a partner. IPG will reject duplicated transmission.

Method properties

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGPurchase	String	Mandatory	Name of the method requested for execution from IPG.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature.
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	string	Mandatory	Version of protocol used for transition.
Language	EN	A(2)	Mandatory	ISO 2-character code for the desired language on the payment page. If IPG cannot fulfill the requested language, it will set the English language as default. Currently supporting EN, FR, DE, BG, ES, RO, EL, IT, PL.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
BannerIndex	1	Int	Mandatory	Index specified in IPG for every banner provided by the Merchant. The Merchant may choose to select a proper banner for every payment. The banner is displayed on the payment page.
PostResultAction	Redirect	String	Conditional	Values of the parameter: **Redirect** – redirect to URL_OK and URL_Cancel **CloseWindow** – close the window after the transaction and merchant have control of next actions If there is no value in this parameter, iCard will show iCard page for the result of transaction and after that it will redirect to merchant site.
MID	000000000000123	AN(15)	Mandatory	Identifier of the virtual terminal used for the purchase.

MIDName	Merchant Web Shop	String	Mandatory	User friendly name of the merchant web shop. The cardholder will see this name on some notices in the payment page.
Amount	23.45	Double (8,2)	Mandatory	The amount of the payment requested.
Currency	978	N (3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
CustomerIP	127.0.0.1	String	Mandatory	Customer IP address in IPv4 or IPv6 format
OrderID	60EC4A03-0AC1-44E5-872B-08FC5AAC9FDB	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.
CustomerIdentifier	1234	String	Recommended	Credentials of the customer at merchant platform (ID, phone number or names).
Email	customer@mywebsite.com	String	Mandatory	This is the cardholder's email.
URL_Notify	http://site.ext/paymentNotify	String	Mandatory	Address supplied by the partner, where iPG API will send the callbacks.
URL_OK	http://site.ext/paymentOK	String	Conditional	The page where the cardholder should be redirected on successful payment. Do not required for PostResultAction = CloseWindow.
URL_Cancel	http://site.ext/paymentNOK	String	Conditional	The page where the cardholder should be redirected when the transaction is unsuccessful. Do not required for PostResultAction = CloseWindow.
Note	Note	String	Optional	Text associated with the purchase.
IPGPaymentContext	GooglePay	String	Conditional	Context of the payment process with tokenized card - GooglePay.
Signature	Byte[]	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Apple Pay JS SDK with redirect Implementation

Purpose

Apple Pay is mobile payment and digital wallet service from Apple, which provides secure and user-friendly solution for users to pay on the merchant website. iCard has integration with Apple that provides two different implementations for the gambling businesses model which can be implemented in the merchant website. In this chapter is described **JS SDK with redirect** solution.

JS SDK is used for checking the availability of the **APay** button on the device of the cardholder. Merchant uses **iCard redirect** for payment checkout and transaction process.

Additional Details

Apple Pay is available only if the user's card is tokenized and if they use a device which allows user authentication. These are the requirements for incorporating Apple Pay on your website:

- All pages that include Apple Pay must be served over HTTPS.
- Merchant's domain must have a valid SSL certificate.
- Merchant's server must support the Transport Layer Security (TLS) protocol version 1.2.

Merchant severs set ups

Allow Apple Pay IP Addresses

To successfully connect with Apple Pay IP addresses and domains, your server needs to allow access over HTTPS (TCP over port 443) and include the TLS Server Name Indication (SNI) extension. Apple Pay requires SNI on all connections. Your server needs to connect to the Apple Pay IP addresses and domains provided listed in Setting Up Your Server, below.

Allow Apple IP Addresses for Domain Verification

Apple uses the following IP addresses when you register or verify your merchant domain. If you protect your domain from public access and you wish to complete domain verification, you need to allow the following IP address ranges.

```
17.32.139.128/27
17.32.139.160/27
17.140.126.0/27
17.140.126.32/27
17.179.144.128/27
17.179.144.160/27
17.179.144.192/27
17.179.144.224/27
17.253.0.0/16
```

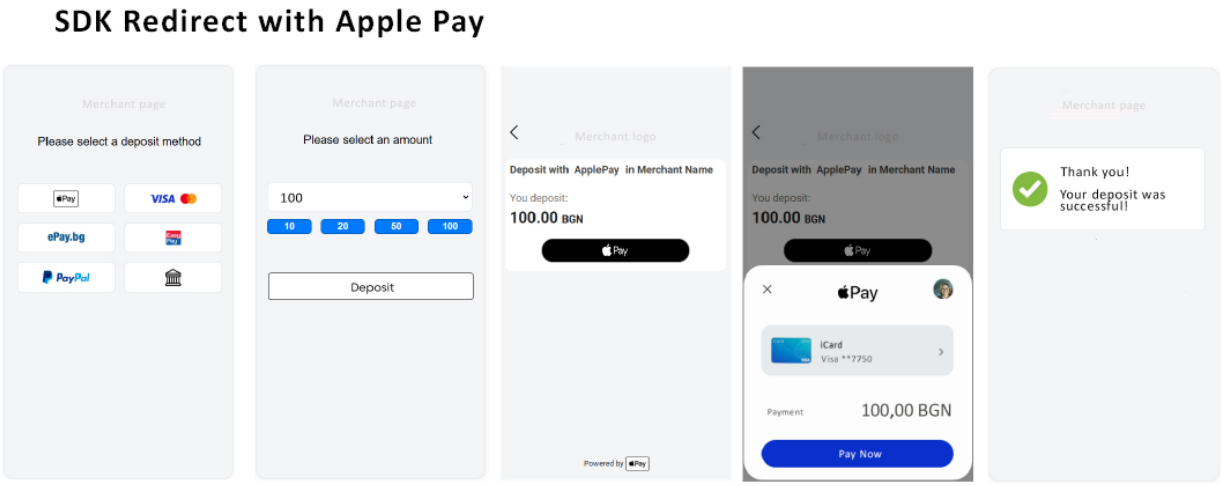
Configuring Merchant – iCard – Apple Environment

First step which must be completed by the Merchant during the registration process is to host iCard's domain verification file on the Merchant's domain. This is applicable for **both sandbox and production environments**. The file will be provided on demand from cs.integrations@icard.com .

- The domain-verification files must be hosted at the following path for each domain that's being registered:
  https://[DOMAIN_NAME]/.well-known/apple-developer-merchantid-domain-association

User Flow details

- **Payment Method Selection** – When the user is browsing available deposit methods on the merchant page.
- **Pre – Deposit Screen** – Displayed when the user is about to make a deposit on the merchant page.
- **Payment Method Screen** – Shown when the user selects Apple Pay as their payment method on the iCard redirect checkout.
- **Apple Pay API Payment Screen** – Displays the payment information the user has saved in Apple Pay.
- **Post – Deposit Screen** – Shown after the user successfully completes a deposit to the merchant page.



Apple payment method on merchant page

Payment Method Selection page

In order to display Apple Pay as a payment method, a Merchant should include the following script on their page:

HTML head section

Sandbox environment:

```
<head>
        <script src="https://dev-ipg.icards.eu/sandbox/js/icard-g-a-pay.min.js"></script>
</head>
```

Production environment:

```
<head>
        <script src="https://ipg.icard.com/js/icard-g-a-pay.min.js"></script>
</head>
```

In order to check for **APay** availability, use the following example script:

```
<script>
  window.addEventListener('DOMContentLoaded', function () {
    const config = {
      mid: merchant id (provided by iCard), //type string
      environment: 'prod/sandbox'
    };

    const applePay = new ICardIpgGAPay(config)
    googlePay.isGooglePayAvailable()
        .then((isReady) => {
          if (isReady) {
            //Show Apple payment method btn
          }
        })
        .catch((error) => {
          console.log(error);
        })
  })
</script>
```

Purchase with Apple Pay (API call: IPGPurchase)

On page **Pre – Deposit Screen** button Deposit is in control of the Merchant. Next step after push Deposit from client Merchant should redirect to IPG with payment request described in **Payment method Screen** with method properties:

Method properties

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGPurchase	String	Mandatory	Name of the method requested for execution from IPG.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature.
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	string	Mandatory	Version of protocol used for transition.
Language	EN	A(2)	Mandatory	ISO 2-character code for the desired language on the payment page. If IPG cannot fulfill the requested language, it will set the English language as default. Currently supporting EN, FR, DE, BG, ES, RO, EL, IT, PL.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
BannerIndex	1	Int	Mandatory	Index specified in IPG for every banner provided by the Merchant. The Merchant may choose to select a proper banner for every payment. The banner is displayed on the payment page.
PostResultAction	Redirect	String	Conditional	Values of the parameter: **Redirect** – redirect to URL_OK and URL_Cancel **CloseWindow** – close the window after the transaction and merchant have control of next actions If there is no value in this parameter, iCard will show iCard page for the result of transaction and after that it will redirect to merchant site.
MID	000000000000123	AN(15)	Mandatory	Identifier of the virtual terminal used for the purchase.
MIDName	Merchant Web Shop	String	Mandatory	User friendly name of the merchant web shop. The cardholder will see this name on some notices in the payment page.
Amount	23.45	Double (8,2)	Mandatory	The amount of the payment requested.
Currency	978	N (3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
CustomerIP	127.0.0.1	String	Mandatory	Customer IP address in IPv4 or IPv6 format

OrderID	60EC4A03-0AC1-44E5-872B-08FC5AAC9FDB	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.
CustomerIdentifier	1234	String	Recommended	Credentials of the customer at merchant platform (ID, phone number or names).
Email	customer@mywebsite.com	String	Mandatory	This is the cardholder's email.
URL_Notify	http://site.ext/paymentNotify	String	Mandatory	Address supplied by the partner, where iPG API will send the callbacks.
URL_OK	http://site.ext/paymentOK	String	Conditional	The page where the cardholder should be redirected on successful payment. Do not required for PostResultAction = CloseWindow.
URL_Cancel	http://site.ext/paymentNOK	String	Conditional	The page where the cardholder should be redirected when the transaction is unsuccessful. Do not required for PostResultAction = CloseWindow.
Note	Note	String	Optional	Text associated with the purchase.
IPGPaymentContext	ApplePay	String	Conditional	Context of the payment process with tokenized card - ApplePay.
Signature	Byte[]	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Apple Pay API Payment Screen and Post – Deposit Screen

These sections are processed in the iCard hosted page. The user flow ends with approved or declined transaction.

Apple Pay only JS SDK Implementation

Purpose

Apple Pay is mobile payment and digital wallet service from Apple, which provides secure and user-friendly solution for users to pay on the merchant website. iCard has integration with Apple that provides two different implementations for the gambling businesses model which can be implemented in the merchant website. In this chapter is described **JS SDK** solution.

JS SDK is used on the pages:

- Choose payment methos page – for checking the availability of the APay button on the device of the cardholder.
- Deposit Page – to display APay button and process payment

Additional Details

Apple Pay is available only if the user's card is tokenized and if they use a device which allows user authentication. These are the requirements for incorporating Apple Pay on your website:

- All pages that include Apple Pay must be served over HTTPS.
- Merchant's domain must have a valid SSL certificate.
- Merchant's server must support the Transport Layer Security (TLS) protocol version 1.2.

Merchant severs set ups

Allow Apple Pay IP Addresses

To successfully connect with Apple Pay IP addresses and domains, your server needs to allow access over HTTPS (TCP over port 443) and include the TLS Server Name Indication (SNI) extension. Apple Pay requires SNI on all connections. Your server needs to connect to the Apple Pay IP addresses and domains provided listed in Setting Up Your Server, below.

Allow Apple IP Addresses for Domain Verification

Apple uses the following IP addresses when you register or verify your merchant domain. If you protect your domain from public access and you wish to complete domain verification, you need to allow the following IP address ranges.

```
17.32.139.128/27
17.32.139.160/27
17.140.126.0/27
17.140.126.32/27
17.179.144.128/27
17.179.144.160/27
17.179.144.192/27
17.179.144.224/27
17.253.0.0/16
```

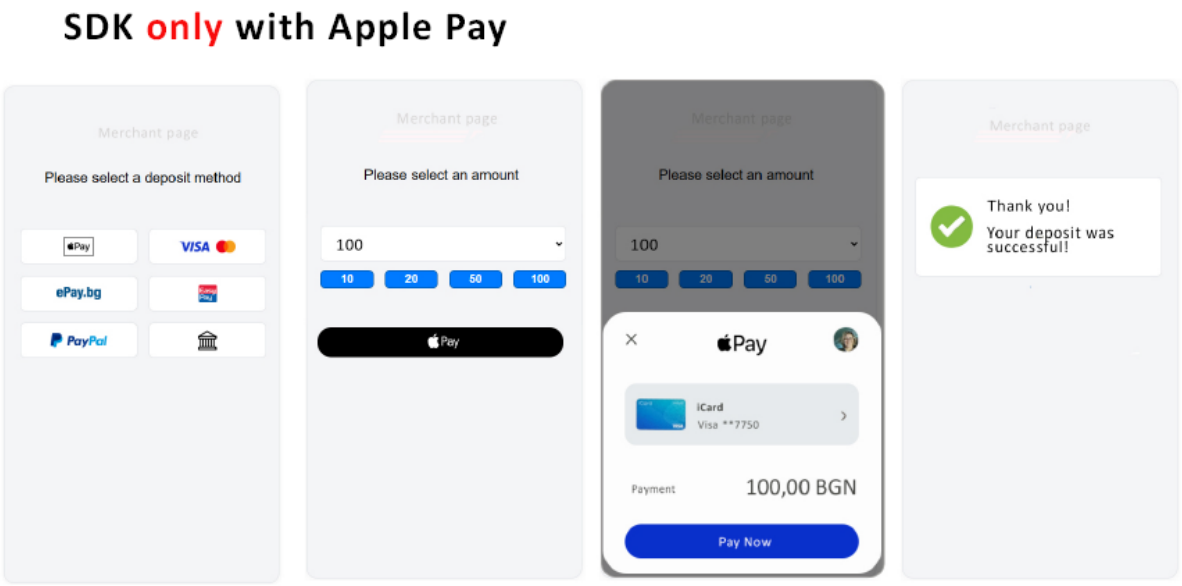
Configuring Merchant – iCard – Apple Environment

First step which must be completed by the Merchant during the registration process is to host iCard's domain verification file on the Merchant's domain. This is applicable for **both sandbox and production environments**. The file will be provided on demand from cs.integrations@icard.com .

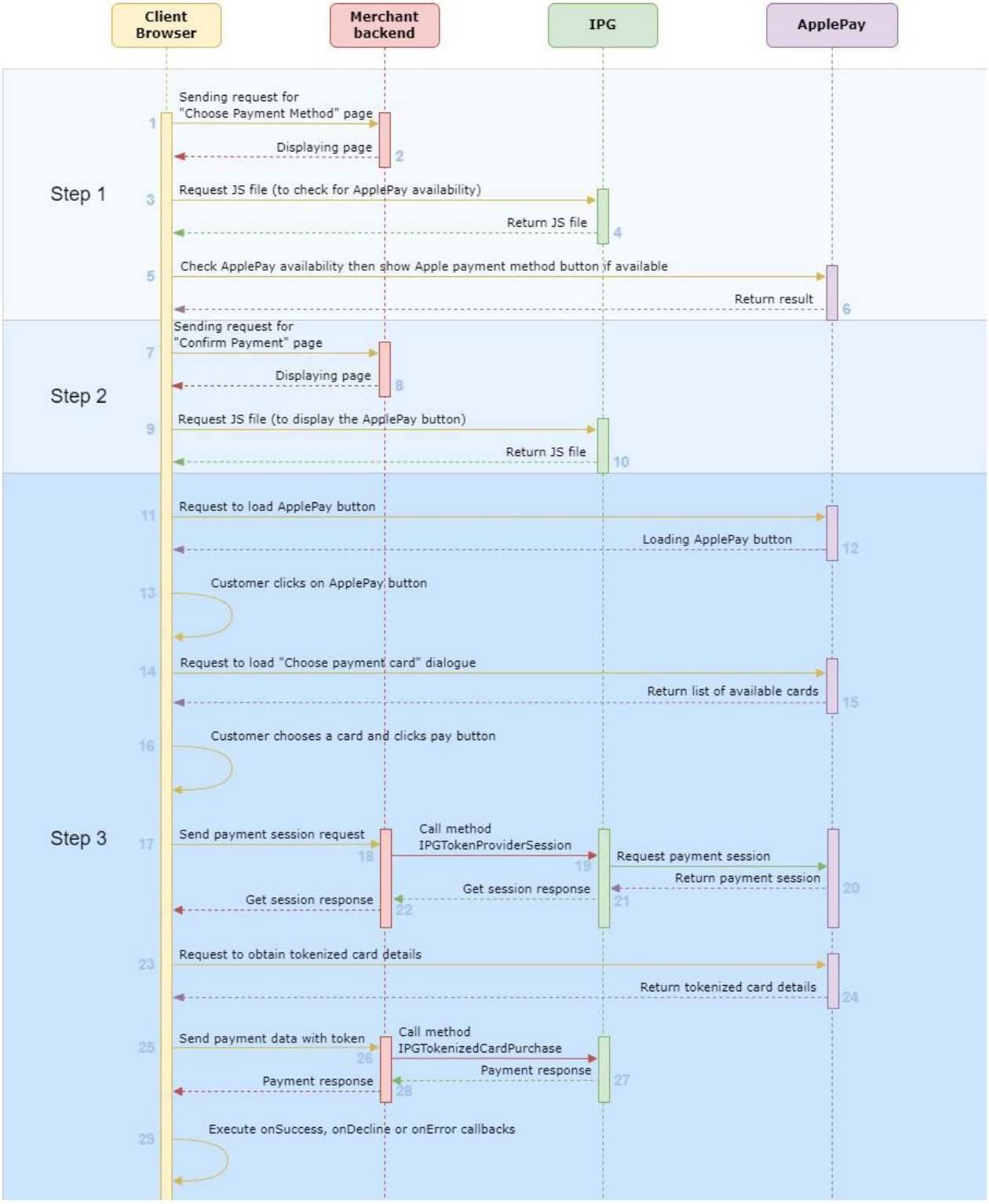
- The domain-verification files must be hosted at the following path for each domain that's being registered:
  https://[DOMAIN_NAME]/.well-known/apple-developer-merchantid-domain-association

User Flow details

- **Payment Method Selection** – When the user is browsing available deposit methods on the merchant page.
- **Deposit Page** – Displayed when the user is about to make a deposit on the merchant page.
- **Apple Pay API Payment Screen** – Displays the payment information the user has saved in Apple Pay.
- **Post – Deposit Screen** – Shown after the user successfully completes a deposit to the merchant page.



Process flow of transaction with Apple Pay only SDK:



Apple payment method on merchant page

Payment Method Selection

In order to display Apple Pay as a payment method, a Merchant should include the following script on their page:

HTML head section

Sandbox environment:

```
<head>
        <script src="https://dev-ipg.icards.eu/sandbox/js/icard-g-a-pay.min.js"></script>
</head>
```

Production environment:

```
<head>
        <script src="https://ipg.icard.com/js/icard-g-a-pay.min.js"></script>
</head>
```

In order to check for **APay** availability, use the following example script:

```
<script>
  window.addEventListener('DOMContentLoaded', function () {
    const config = {
      mid: merchant id (provided by iCard), //type string
      environment: 'prod/sandbox'
    };

    const applePay = new ICardIpgGAPay(config)

    googlePay.isGooglePayAvailable()
        .then((isReady) => {
          if (isReady) {
            //Show Apple payment method btn
          }
        })
        .catch((error) => {
          console.log(error);
        })
  })
</script>
```

Deposit Page

In order to display the Apple Pay button on the checkout page, a Merchant should include the following script on their page:

HTML head section

Sandbox environment:

```
<head>
<script src="https://dev-ipg.icards.eu/sandbox/js/icard-g-a-pay.min.js"></script>
</head>
```

Production environment:

```
<head>
	<script src="https://ipg.icard.com/js/icard-g-a-pay.min.js"></script>
</head>
```

Example script to display the Apple Pay button:

```
<script>
	window.addEventListener('DOMContentLoaded', function () {
		const config = {
			processPaymentUrl: <merchant endpoint to accept payment token>,
			mid: <MID provided by iCard>, //type string
			merchantName: <name of the merchant displayed on the Payment Page>,
			amount: <payment amount>, //type string
			currencyAlpha: <currency alpha code (BGN, EUR…)>,
			onSuccess: function (successData) {
				console.log(successData);
				alert('Success');
				//merchants can use this callback to handle successful payment
			},
			onDecline: function (declineData) {
				console.log(declineData);
				alert('Failed payment');
				//merchants can use this callback to handle unsuccessful payment
			},
		onError: function (errorData) {
				console.log(errorData);
				alert('Failed payment');
				//merchants can use this callback to handle payment errors
			},

			environment: 'sandbox', //environments: prod, sandbox;
			appleConfig: {
				btnContainerId: <ID of element which contains the Apple Pay button>,
				btnColor: 'black', //possible values – black, white, white-outline
				merchantDomain: <Verified domain in Section 2.2>
			},
		googleConfig: {
			btnContainerId: <ID of element which contains the Google Pay button>,
			btnColor: 'black/white'
	},
			merchantSessionData: <additional optional values (order ID, session data, etc.)>
		};

		const googleApplePay = new ICardIpgGAPay(config)
		googleApplePay.create();

		//if there is a functionality where the customer can change amount on checkout page, then
		//the following function should be used:
		googleApplePay.ipgSetAmount(new amount value casted to string);
	})
</script>
```

Apple Pay API Payment Screen

To initiate the Apple Pay process, a session must be obtained from Apple. For this purpose, our **SDK** sends an **AJAX** request from client browser to merchant backend for **Get Apple Pay session with Token Provider session in APICall: IPGTokenProviderSession.**

Method request properties IPGTokenProviderSession

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGTokenProviderSession	String	Mandatory	Name of the method requested for execution from IPG.
Amount	10.48	Double (8,2)	Mandatory	Payment amount
MerchantUrl	dev-ipg.icards.eu	String	Mandatory	Merchant domain
ValidationURL	https://apple-paygateway …	String	Mandatory	Provided by Apple
DisplayName	My store	String	Mandatory	Merchant store name
TokenizedCardProvider	Apple	String	Mandatory	
MerchantSessionData	Any data	Any type	Optional	Provided by the merchant.

After the merchant receives the data from the **AJAX request IPGTokenProviderSession**, a new backend request is prepared to the IPG API with the following data::

Method request properties from merchant backend to IPG platform IPGTokenProviderSession

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGTokenProviderSession	String	Mandatory	Name of the method requested for execution from IPG.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	String	Mandatory	Version of protocol used for transition.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
OutputFormat	json	String	Mandatory	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.
MID	000000000000123	AN(15)	Mandatory	Identifier of the virtual terminal used for the purchase.
OrderID	60EC4A03-0AC1-44E5-872B-08FC5AAC9FDB	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.
MerchantUrl	dev-ipg.icards.eu	String	Mandatory	Merchant domain

ValidationURL	https://apple-paygateway …	String	Mandatory	Provided by Apple
DisplayName	My store	String	Mandatory	Merchant store name
TokenizedCardProvider	Apple	String	Mandatory	
Signature	Byte[]	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Method response properties

Property	Typical value	Type	Description
IPGmethod	IPGTokenProviderSession	String	Name of the method
OrderID	F989C51B-A967-4396-A932-109D4238A21A	String	OrderID from request data
Status	0	String	Request status. Refer to document IPG Additions Status Codes
StatusMsg	Success	String	Status message. Refer to document IPG Additions Status Codes
Session	ApplePay session data	String	Session from ApplePay to process payment
Signature	Byte[]	BASE64	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

After Merchant received the response, the following actions are:

In case of invalid Signature, the Merchant must respond to the AJAX request with HTTP status code 401. If the Signature is valid, then the Merchant must respond with data from method response.

After the user confirms their payment in **Apple Pay from the browser**, an AJAX request is sent to the merchant's backend with the following data in **APICall: IPGTokenizedCardPurchase**

Method request properties IPGTokenizedCardPurchase

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGTokenizedCardPurchase	String	Mandatory	Name of the method requested for execution from IPG.
Amount	10.48	Double (8,2)	Mandatory	Payment amount
TokenizedCard		String	Mandatory	Encrypted card data.
TokenizedCardProvider	Apple/Google	String	Mandatory	
MerchantSessionData	Any data	Any type	Optional	Provided by the merchant. In case of ApplePay this data contains OrderID from previous IPGTokenProviderSession request. **This OrderID should be send in IPGTokenizedCardPurchase request.**

After the merchant receives the data from the AJAX request **IPGTokenizedCardPurchase**, a new backend request is prepared to the IPG API with the following data:

Method request from merchant backend to IPG platform IPGTokenizedCardPurchase:

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGTokenizedCardPurchase	String	Mandatory	Name of the method requested for execution from IPG.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	String	Mandatory	Version of protocol used for transition.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
OutputFormat	json	String	Mandatory	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.
MID	000000000000123	AN(15)	Mandatory	Identifier of the virtual terminal used for the purchase.
OrderID	60EC4A03-0AC1-44E5-872B-08FC5AAC9FDB	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the

				request that will help the merchant to recognize for which order is the payment. **In case of ApplePay this OrderID should be same as IPGTokenProviderSession request**
Email	customer@test.com	String	Mandatory	This is the cardholder's email.
CustomerIdentifier	1234	String	Recommended	Credentials of the customer at merchant platform (ID, phone number or names).
Amount	10.48	Double (8,2)	Mandatory	Payment amount
Currency	978	N(3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
URL_Notify	http://site.ext/paymentNotify	String	Mandatory	Address supplied by the partner, where iPG API will send the callbacks.
TokenizedCardProvider	Apple	String	Mandatory	
TokenizedCard		String	Mandatory	Encrypted card data
Signature	Byte[]	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Method response properties

Property	Typical value	Type	Description
IPGmethod	IPGTokenizedCardPurchase	String	Name of the method
OrderID	F989C51B-A967-4396-A932-109D4238A21A	String	OrderID from request data
Status	0	String	Request status. Refer to document IPG Additions Status Codes
StatusMsg	Success	String	Status message. Refer to document IPG Additions Status Codes
Signature	Byte[]	BASE64	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

After Merchant received the response, the following actions are:

In case of invalid Signature, the Merchant must respond to the AJAX request with HTTP status code 401. If the Signature is valid, then the Merchant must respond with data from method response.

Backend Implementation

Payout through Processing Original Credit Transaction (OCT) (API call: IPGOCT)

Purpose

This method is used by IPG to allow merchant to process OCT transaction. OCT transactions are used for Gaming Withdrawal.

This method is intended to be utilized by the Merchant in their website back-end. IPG API will return a response with the result. The request can be initialized only by a reference from a previously executed payment transaction or by a token of a previous executed stored card.

IPG will check:

- Valid MID.
- Valid currency with regards to the MID.

Method request properties IPGOCT

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGOCT	String	Mandatory	Name of the method requested for execution from IPG.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	String	Mandatory	Version of protocol used for transition.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
MID	000000000000123	AN (15)	Mandatory	Identifier of the virtual terminal used for the purchase.
OrderID	610F0A8D-7210-4828-B625-C02E843DE7D8	String (50)	Mandatory	Placeholder for the merchant. Used for unique ID of the request that will help the merchant to recognize for which order is the payment.
IPG_Trnref	20250602110038002328	String	Conditional	**Mandatory only for OCT by TRN and Approval**. Used to uniquely identify a transaction in IPG. Used as a unique parameter in iCard system.

Approval	123456	String	Conditional	**Mandatory only for OCT by TRN and Approval.** Approval code return by the issuer of the card. Used to identify the financial transaction within the card schemes and the issuer.
CardToken	40B1B011C4A21EA65A8AA06E9D767ECE348ADB2E2D4E4E6C3A0536E452619059	String	Conditional	**Mandatory only for OCT by Token of the card**. Token of the card is received in Notify callback in case client choose checkbox - Save card during IPGPurchase. **Used instead of parameter IPG_Trnref.**
Amount	23.45	Double	Mandatory	The amount of the payment requested.
Currency	978	N (3)	Mandatory	ISO numeric currency code. The currency for the payment must be equal to the currency of the MID.
RecipientFirst Name	John	String (35)	Mandatory	First Name of the recipient. Must not contain all spaces, all zeros, all numerics, or question marks.
RecipientLast Name	Smith	String (35)	Mandatory	Last Name of the recipient. Must not contain all spaces, all zeros, all numerics, or question marks.
OutputFormat	json	String	Optional	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.
Signature	uIkMPIYhgE.....7DEhXOaUKakY=	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST**, as it is not used to calculate the hash.

Method response properties

Property	Typical value	Type	Description
IPGmethod	IPGOCT	String	Name of the method
OrderID	610F0A8D-7210-4828-B625-C02E843DE7D8	String	Echo from IPGOCT
IPGTrnref	20250602110038002328	String	Used to uniquely identify a transaction in IPG.
IPGTrnrefOriginal	20250602110038002328	String	Transaction ID of original transaction. Available if execute **OCT by IPG_Trnref and Approval**.
Status	0	String	Request status. Refer to document IPG Additions Status Codes.
StatusMsg	Success	String	Status message. Refer to document IPG Additions Status Codes
Signature	uIkMPIYhgE.....7DEhXOaUKakY=	BASE64	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Get transaction status for previously executed payment (API call: IPGGetTxnStatus)

Purpose

This method is used by Merchant to get the current status of a previously executed payment for **IPGOCT**. The IPG API will return an xml or json with information about a specific OrderID. This method is intended to be utilized by the Merchant in his website back-end. The Merchant could decide whether or not to use this method. We recommend using it only in case of an **OCT timeout**.

NB! Please, bear in mind that this functionality has a reference purpose. It shouldn't be mistaken with message "Transaction is approved". A transaction is approved/passed successfully, when the following conditions are met:

- **IPGTrnStatus –** 0
- **IPGTrnStatusMsg –** Transaction completed successful

Method request properties

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGGetTxnStatus	String	Mandatory	Name of the method requested for execution from IPG.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	String	Mandatory	Version of protocol used for transition.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
MID	000000000000123	AN (15)	Mandatory	Identifier of the virtual terminal used for the purchase.
OrderID	610F0A8D-7210-4828-B625-C02E843DE7D8	String (50)	Mandatory	Placeholder for the merchant. Used to put some data that will help the merchant to recognize for which order is the payment.
OutputFormat	json	String	Optional	Output format of data. The property can be "xml" or "json". If it is not specified in the request, the default value is "xml".
Signature	uIkMPIYhgE.....7DEhXOaUKakY=	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Method response properties

Property	Typical value	Type	Description
IPGmethod	IPGGetTxnStatus	String	Name of the method
Status	0	String	Request status. Refer to document IPG Additions Status Codes.
StatusMsg	Success	String	Status message. Refer to document IPG Additions Status Codes.
OrderID	610F0A8D-7210-4828-B625-C02E843DE7D8	String	Echo from IPGGetTxnStatus
IPGTrnStatus	0	Int	Transaction Request status. Refer to document IPG Additions Status Codes.
IPGTrnStatusMsg	Success	String	Transaction Status message. Refer to document IPG Additions Status Codes.
Signature	uIkMPIYhgE.....7DEhXOaUKakY=	BASE64	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST**, as it is not used to calculate the hash.

Make a reversal for previously executed payment (API call: IPGReversal)

Purpose

This method is used by Merchant to initiate a reversal of previously executed payment. The IPG API will return an xml or json with the result. This method is intended to be utilized by the Merchant in his website back-end. This method is **mandatory** for integration for all merchants.

Method properties

Property	Typical value	Type	Requirement status	Description
IPGmethod	IPGReversal	String	Mandatory	Name of the method requested for execution from IPG.
KeyIndex	1	Int	Mandatory	Identifier of the private key used for signature
KeyIndexResp	1	Int	Mandatory	Identifier of the private key used to build signature for response to merchant
IPGVersion	4.5	String	Mandatory	Version of protocol used for transition.
Originator	33	Int	Mandatory	Value that uniquely identifies the merchant company that has signed a contract with iCard AD.
OutputFormat	json	String	Mandatory	Output format of data. The property can be “xml” or “json”. If it is not specified in the request, the default value is “xml”.
OrderID	60EC4A03-0AC1-44E5-872B-08FC5AAC9FDB	String	Mandatory	OrderID of the previous transaction that should be reversed.
MID	000000000000123	AN(15)	Mandatory	MID of the previous transaction that should be reversed.
IPG_Trnref	202504170836273 62872	String	Mandatory	Used to uniquely identify a transaction in IPG. Used as a parameter for subsequent refund of reversal if needed.
Signature	Byte[]	BASE64	Mandatory	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**

Method response properties

Property	Typical value	Type	Description
IPGmethod	IPGReversal	String	Name of the method.
OrderID	F989C51B-A967-4396-A932-109D4238A21A	String	OrderID from request data.
IPGTrnref	20250417083627362872	String	Used to uniquely identify a transaction in IPG.
IPGTrnrefOriginal	20250602110038002328	String	Transaction ID of original transaction.
Status	0	String	Request status. Refer to document IPG Additions Status Codes.
StatusMsg	Success	String	Status message. Refer to document IPG Additions Status Codes.
Signature	Byte[]	BASE64	Signed HASH for all properties in the command. Signature is **ALWAYS THE LAST PARAMETER IN THE POST,** as it is not used to calculate the hash**.**